

09:00-09:50 (쉬는 시간 11:30-12:00)

PR Target과 Fixture

PR · Commit · pr_target.json

진행

PR Target과 Fixture
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

data/day4_github/pr_fixture.json
src/course_services/github_service.py

확인할 결과

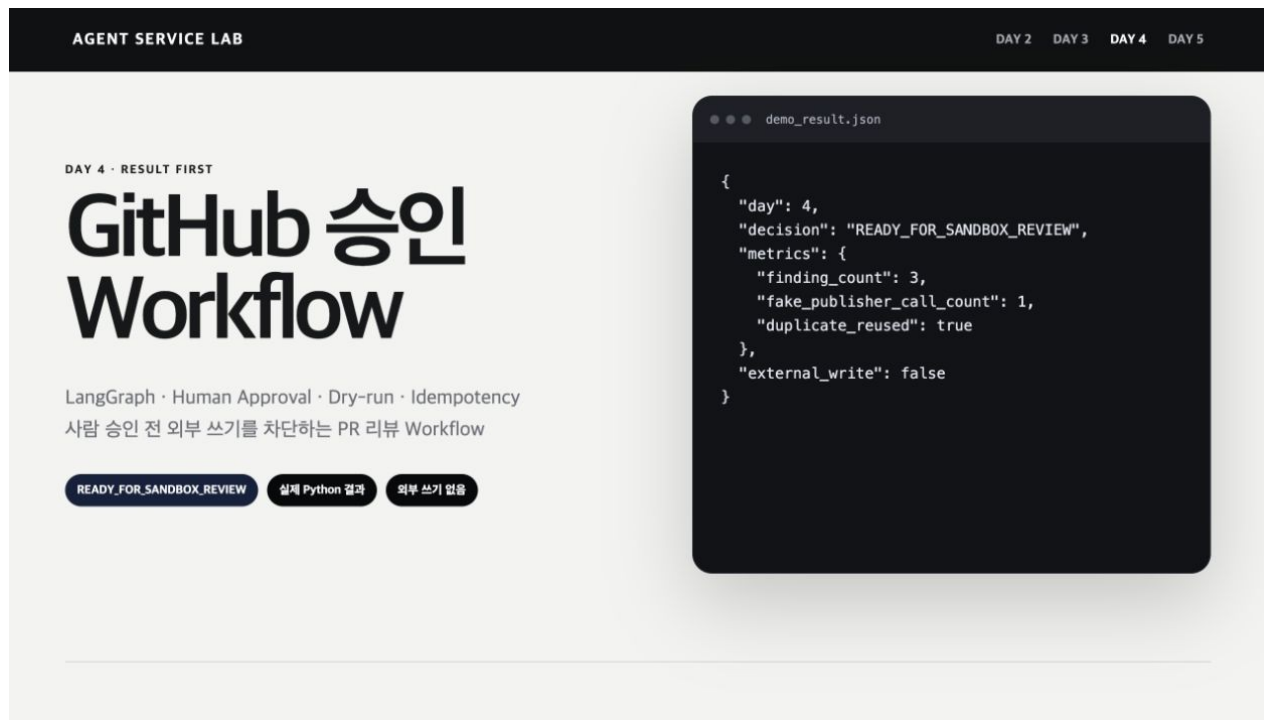
pr_target.json

PR Review Automation Service 완성 화면

대상 commit이 바뀌면 이전 line mapping과 review comment가 stale해진다.

입력 계약

- 핵심 처리
- 실패 경계
- 사람 판단
- 평가·실행 증거



재생 파일 · assets/demo-videos/day4_service_teaser.gif

오늘 8차시는 이 화면을 현재 저장소에서 다시 만드는 과정입니다.

4일차 전체 시간표

3시간 블록(30분 휴식) · 2시간 블록(20분 휴식) · 점심 14:00-15:00 · 3시간 블록(30분 휴식)

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	PR Target과 Fixture	강의 12 시연 10 Codex 제작 23 확인 5분	pr_target.json
09:50-10:40	2차시	GitHub 인증과 최소 권한	강의 12 시연 10 Codex 제작 23 확인 5분	.env.sample
10:40-11:30	3차시	REST 상태 코드와 복구	강의 12 시연 10 Codex 제작 23 확인 5분	github_read_result.json
12:00-12:50	4차시	LangGraph State와 분기	강의 12 시연 10 Codex 제작 23 확인 5분	compiled_review_graph
12:50-13:40	5차시	Checkpoint와 Idempotency	강의 12 시연 10 Codex 제작 23 확인 5분	checkpoint_and_audit.json
15:00-15:50	6차시	Human Approval과 Interrupt	강의 12 시연 10 Codex 제작 23 확인 5분	review_decision.json
15:50-16:40	7차시	Dry-run Comment Payload	강의 12 시연 10 Codex 제작 23 확인 5분	review_comment_plan.json
16:40-17:30	8차시	Sandbox GitHub 게시	강의 12 시연 10 Codex 제작 23 확인 5분	day4_audit_record.json

11:30-12:00 쉬는 시간 · 13:40-14:00 쉬는 시간 · 14:00-15:00 점심시간 · 17:30-18:00 쉬는 시간

PR Target과 Fixture 학습 결과

- 01 — repository·PR number·head SHA·diff를 fixture에서 읽는다.
- 02 — 화면에 보이는 PR 제목만 믿고 repo·number·SHA를 확인하지 않는다. 왜 위험한지 설명한다.
- 03 — pr_target.json에서 정상과 실패 상태를 직접 확인한다.

PR Target과 Fixture 용어

용어	이 수업에서 쓰는 뜻
PR	변경을 검토·병합하는 요청
Commit	특정 시점의 변경 묶음
Head SHA	PR 최신 commit 식별자
Review comment	diff line에 붙는 의견

pr_target.json 입출력

INPUT

data/day4_github/pr_fixture.json

OUTPUT

pr_target.json

PROCESS

fixture load → target validate → head SHA 고정

VERIFY

synthetic fixture가 PullRequestTarget으로 validate된다.
필드 누락이나 workspace 밖 fixture는 구조화 오류다.

PR Target과 Fixture 실행 흐름

01

fixture load

02

target validate

03

head SHA 고정

04

diff load

05

read-only result

마지막 판단은 pr_target.json에서 사람이 확인합니다.

pr_target.json 정상·실패 계약

정상	——	synthetic fixture가 PullRequestTarget으로 validate된다.
경계	——	필드 누락이나 workspace 밖 fixture는 구조화 오류다.
외부 쓰기	——	PR Target과 Fixture: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·pr_target.json

PR Target과 Fixture 파일 지도

입력·fixture

data/day4_github/pr_fixture.json

구현 코드

src/course_services/github_service.py

검증·test

data/day3_review_cases/unsafe_pr.diff

따라하기 notebook

materials/day4/day4_service_lab.ipynb

PR Target과 Fixture 개념·코드

PR

변경을 검토·병합하는 요청

```
data/day4_github/pr_fixture.json
```

Head SHA

PR 최신 commit 식별자

```
data/day3_review_cases/unsafe_pr.diff
```

Commit

특정 시점의 변경 묶음

```
src/course_services/github_service.py
```

Review comment

diff line에 붙는 의견

```
materials/day4/day4_service_lab.ipynb
```

PR Target과 Fixture 선택 기준

구분	역할	이번 차시의 판단 기준
PR	fixture load 단계의 입력 기준	결정론 test로 먼저 확인
Commit	target validate 단계의 교체 경계	adapter 경계를 확인
Head SHA	head SHA 고정 단계의 운영 조건	비용·지연·권한을 확인
Review comment	diff load 단계의 판단 기록	사람 판단과 운영 기록을 확인

PR Target과 Fixture 대표 실패

실패

화면에 보이는 PR 제목만 믿고
repo·number·SHA를 확인하지 않는다.

사용자 영향

화면에 보이는 PR 제목만 믿고
repo·number·SHA를 확인하지 않는다.
상태에서는 pr_target.json을 운영 판단에 사용할
수 없다.

PR Target과 Fixture 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

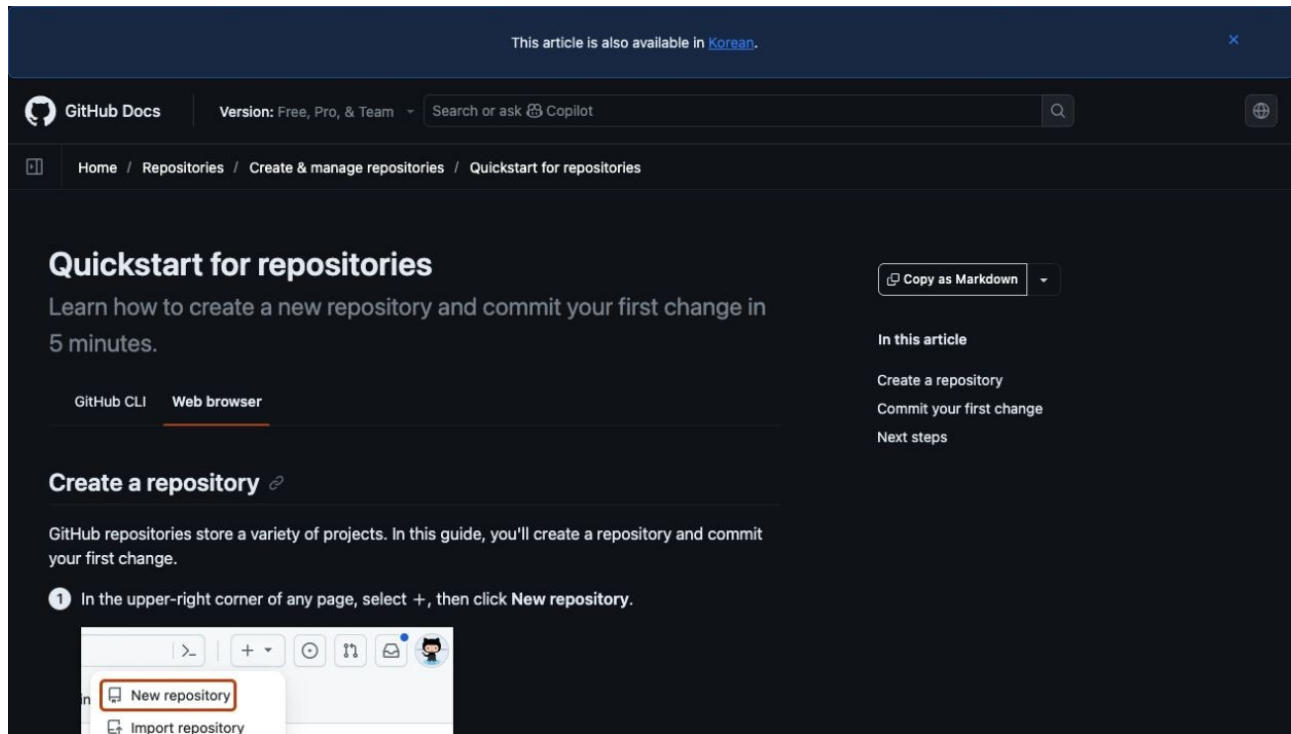
PR Target과 Fixture 복구 경로

- 01 — 탐지: 필드 누락이나 workspace 밖 fixture는 구조화 오류다.
- 02 — 중단: 화면에 보이는 PR 제목만 믿고 repo·number·SHA를 확인하지 않는다.
- 03 — 복구: 쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.
- 04 — 증거: pr_target.json과 test 결과를 함께 남긴다.

PR Target과 Fixture Codex 검증 루프

src/course_services/
github_service.py의 책임을 찾고
pr_target.json 계약을 focused
test로 고정합니다.

PR Target과 Fixture
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



PR Target과 Fixture Codex 작업 명세

목표

fixture loader에 필수 field·workspace path test를 추가한다.

허용 경로

src/course_services/github_service.py
data/day3_review_cases/unsafe_pr.diff

완료 조건

synthetic fixture가 PullRequestTarget으로 validate된다.
필드 누락이나 workspace 밖 fixture는 구조화 오류다.

금지

PR Target과 Fixture: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

PR Target과 Fixture 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | PR Target과 Fixture 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | synthetic fixture가 PullRequestTarget으로 validate된다. |
| 03 | Claude review | —— | pr_target.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | PR Target과 Fixture의 두 LLM 의견과 test는 자동 승인이 아니다 |

PR Target과 Fixture 정상 시연

- 01 — 열기: `data/day4_github/pr_fixture.json`
- 02 — 구현 확인: `src/course_services/github_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k github`
- 04 — 성공 신호: synthetic fixture가 PullRequestTarget으로 validate된다.

PR Target과 Fixture 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k github
```

실행 전 확인

PR Target과 Fixture: repository root · Python 3.12 · .env 미출력

01_pr_target.json 실행 결과

```
{  
  "repository": "classroom-owner/agent-sandbox",  
  "number": 17,  
  "head_sha": "7c83e8fce9a24d90b6d4a6c9123aa90817b65b1a"  
}
```

PR Target과 Fixture 실패·복구 시연

재현

필드 누락이나 workspace 밖 fixture는 구조화 오류다.

복구

쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.

확인: 필드 누락이나 workspace 밖 fixture는 구조화 오류다.

PR Target과 Fixture 소프트웨어 제작

열 파일

`src/course_services/github_service.py`
`data/day3_review_cases/unsafe_pr.diff`

작업 범위

PR Target과 Fixture 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

synthetic fixture가 PullRequestTarget으로 validate된다.
pr_target.json 저장

PR Target과 Fixture Notebook 셀

materials/day4/day4_service_lab.ipynb · 1차시

```
from src.course_services.github_service import load_pr_fixture

fixture = load_pr_fixture(ROOT / "data/day4_github/pr_fixture.json", workspace_root=ROOT)
target = {key: fixture[key] for key in ("repository", "number", "head_sha")}
save_json("01_pr_target.json", target)
target
```

PR Target과 Fixture Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k github
```

저장 위치

```
output/course-labs/day4/01_pr_target.json
```

synthetic fixture가 PullRequestTarget으로 validate된다.
실패 시 pr_target.json의 status·error_code를 먼저 확인

PR Target과 Fixture 실패 증거

필드 누락이나 workspace 밖 fixture는 구조화 오류다.

```
result_file = "output/course-labs/day4/01_pr_target.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

pr_target.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

PR Target과 Fixture 정상 Case

NORMAL

synthetic fixture가 PullRequestTarget으로 validate된다.

- 01 — data/day4_github/pr_fixture.json 입력과 command가 기록됐는가
- 02 — pr_target.json schema가 validate되는가
- 03 — PR Target과 Fixture focused test가 실제로 통과했는가

PR Target과 Fixture 경계 Case

BOUNDARY

필드 누락이나 workspace 밖 fixture는 구조화 오류다.

- 01 — PR Target과 Fixture: raw traceback 대신 stable error_code가 남는가
- 02 — pr_target.json: 외부 쓰기·자동 메일이 false인가
- 03 — 필드 누락이나 workspace 밖 fixture는 구조화 오류다. 재현이 가능한가

PR Target과 Fixture 현업 적용

사내 repo PR을 읽어 검토 대기열을 만드는 read-only bot

- 01 — PR Target과 Fixture은 합성·비식별 sample로 먼저 실행
- 02 — 사내 repo PR을 읽어 검토 대기열을 만드는 read-only bot의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — pr_target.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — pr_target.json을 운영 검토 자료로 사용

PR Target과 Fixture 포트폴리오 적용

PR Target과 Fixture · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 화면에 보이는 PR 제목만 믿고 repo·number·SHA를 확인하지 않는다.의 사용자 영향을 한 문장으로 설명
- 02 — PR Target과 Fixture 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — fixture loader에 필수 field·workspace path test를 추가한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — pr_target.json과 README 재현 절차를 제출

PR Target과 Fixture 운영 점검

품질

무엇을 synthetic fixture가 PullRequestTarget으로 validate된다.로 정의하는가

복구

화면에 보이는 PR 제목만 믿고 repo·number·SHA를 확인하지 않는다. 발생 시 pr_target.json에서 원인 상태를 확인

안전

PR Target과 Fixture의 사람 승인과 external_write=false 위치

관측

pr_target.json에서 어떤 status·error_code를 보는가

pr_target.json 실행 증거

- 01 — 설명: PR Target과 Fixture이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k github`
- 03 — 증거: pr_target.json과 정상·실패 test 결과가 남아 있다.

다음 차시: GitHub 인증과 최소 권한

DAY 4 · 2차시

09:50-10:40 (쉬는 시간 11:30-12:00)

GitHub 인증과 최소 권한

PAT · Fine-grained · .env.sample

진행

GitHub 인증과 최소 권한
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

.env.sample
.gitignore

확인할 결과

.env.sample

2차시 입력과 결과

입력	————	pr_target.json
이번 차시	————	GitHub 인증과 최소 권한
결과	————	.env.sample
다음 연결	————	REST 상태 코드와 복구

GitHub 인증과 최소 권한 필요성

교육용 token 하나가 실수로 commit되면 저장소 전체 위험으로 번진다.

권한 목록에서 .env.sample까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: .env.sample

GitHub 인증과 최소 권한 학습 결과

- 01 — fine-grained token과 최소 repository permission을 설명한다.
- 02 — token 값을 notebook output이나 screenshot에 노출한다. 왜 위험한지 설명한다.
- 03 — .env.sample에서 정상과 실패 상태를 직접 확인한다.

GitHub 인증과 최소 권한 용어

용어	이 수업에서 쓰는 뜻
PAT	개인 API 접근 토큰
Fine-grained	대상과 권한을 좁힌 token
Secret	소스·로그에 남기면 안 되는 값
Least privilege	필요한 최소 권한만 부여

.env.sample 입출력

INPUT

.env.sample

PROCESS

권한 목록 → sandbox repo → .env local

OUTPUT

.env.sample

VERIFY

.env는 ignore되고 .env.sample만 추적된다.
token이 없으면 401을 재시도하지 않고 실행을 중단한다.

GitHub 인증과 최소 권한 실행 흐름

01

권한 목록

02

sandbox repo

03

.env local

04

.gitignore

05

만료·폐기

마지막 판단은 .env.sample에서 사람이 확인합니다.

.env.sample 정상·실패 계약

정상	—— .env는 ignore되고 .env.sample만 추적된다.
경계	—— token이 없으면 401을 재시도하지 않고 실행을 중단한다.
외부 쓰기	—— GitHub 인증과 최소 권한: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	—— status·error_code·입력 식별자·.env.sample

GitHub 인증과 최소 권한 파일 지도

입력·fixture

.env.sample

구현 코드

.gitignore

검증·test

AGENTS.md

따라하기 notebook

data/day4_github/pr_fixture.json

GitHub 인증과 최소 권한 개념·코드

PAT

개인 API 접근 토큰

```
.env.sample
```

Secret

소스·로그에 남기면 안 되는 값

```
AGENTS.md
```

Fine-grained

대상과 권한을 좁힌 token

```
.gitignore
```

Least privilege

필요한 최소 권한만 부여

```
data/day4_github/pr_fixture.json
```


GitHub 인증과 최소 권한 선택 기준

구분	역할	이번 차시의 판단 기준
PAT	권한 목록 단계의 입력 기준	결정론 test로 먼저 확인
Fine-grained	sandbox repo 단계의 교체 경계	adapter 경계를 확인
Secret	.env local 단계의 운영 조건	비용·지연·권한을 확인
Least privilege	.gitignore 단계의 판단 기록	사람 판단과 운영 기록을 확인

GitHub 인증과 최소 권한 대표 실패

실패

token 값을 notebook output이나 screenshot에 노출한다.

사용자 영향

token 값을 notebook output이나 screenshot에 노출한다. 상태에서는 .env.sample을 운영 판단에 사용할 수 없다.

GitHub 인증과 최소 권한 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

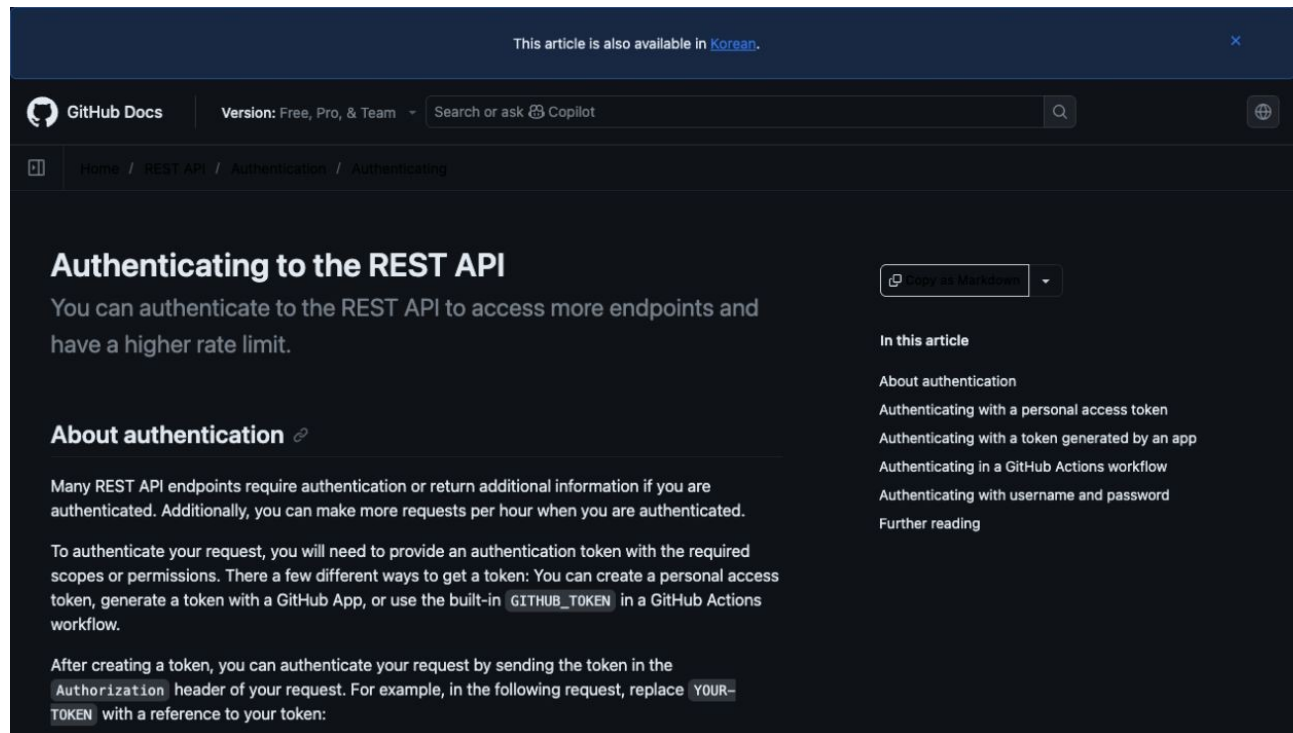
GitHub 인증과 최소 권한 복구 경로

- 01 — 탐지: token이 없으면 401을 재시도하지 않고 실행을 중단한다.
- 02 — 중단: token 값을 notebook output이나 screenshot에 노출한다.
- 03 — 복구: 존재 여부만 bool로 확인하고 노출된 token은 즉시 폐기·재발급한다.
- 04 — 증거: .env.sample과 test 결과를 함께 남긴다.

GitHub 인증과 최소 권한 Codex 검증 루프

.gitignore의 책임을 찾고 .env.sample 계약을 focused test로 고정합니다.

GitHub 인증과 최소 권한
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



GitHub 인증과 최소 권한 Codex 작업 명세

목표

secret을 읽거나 출력하지 않는 범위에서 config validation만 구현한다.

허용 경로

.gitignore
AGENTS.md

완료 조건

.env는 ignore되고 .env.sample만 추적된다.
token이 없으면 401을 재시도하지 않고 실행을 중단한다.

금지

GitHub 인증과 최소 권한: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

GitHub 인증과 최소 권한 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | GitHub 인증과 최소 권한 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | .env는 ignore되고 .env.sample만 추적된다. |
| 03 | Claude review | —— | .env.sample 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | GitHub 인증과 최소 권한의 두 LLM 의견과 test는 자동 승인이 아니다 |

GitHub 인증과 최소 권한 정상 시연

- 01 — 열기: `.env.sample`
- 02 — 구현 확인: `.gitignore`
- 03 — 실행: `git check-ignore -v .env`
- 04 — 성공 신호: `.env`는 ignore되고 `.env.sample`만 추적된다.

GitHub 인증과 최소 권한 실행 명령

```
$ git check-ignore -v .env
```

실행 전 확인

GitHub 인증과 최소 권한: repository root · Python 3.12 · .env 미출력

02_auth_status.json 실행 결과

```
{  
  "env_ignored": true,  
  "github_token_configured": false,  
  "token_value_printed": false,  
  "recommended_scope": "read-only until explicit sandbox approval"  
}
```

GitHub 인증과 최소 권한 실패·복구 시연

재현

token이 없으면 401을 재시도하지 않고
실행을 중단한다.

복구

존재 여부만 bool로 확인하고 노출된
token은 즉시 폐기·재발급한다.

확인: token이 없으면 401을 재시도하지 않고 실행을 중단한다.

GitHub 인증과 최소 권한 소프트웨어 제작

열 파일 **.gitignore**
AGENTS.md

작업 범위 **GitHub 인증과 최소 권한 명세 → 최소 Patch → Focused Test → Diff 검토**

완료 기준 **.env는 ignore되고 .env.sample만 추적된다.**
.env.sample 저장

GitHub 인증과 최소 권한 Notebook 셀

materials/day4/day4_service_lab.ipynb · 2차시

```
import os

ignore_check = run_command("git", "check-ignore", "-v", ".env")
auth_status = {
    "env_ignored": ignore_check["returncode"] == 0,
    "github_token_configured": bool(os.getenv("GITHUB_TOKEN")),
    "token_value_printed": False,
    "recommended_scope": "read-only until explicit sandbox approval",
}
save_json("02_auth_status.json", auth_status)
auth_status
```

GitHub 인증과 최소 권한 Terminal 실행

```
$ git check-ignore -v .env
```

저장 위치

```
output/course-labs/day4/02_auth_status.json
```

.env는 ignore되고 .env.sample만 추적된다.
실패 시 .env.sample의 status·error_code를 먼저 확인

GitHub 인증과 최소 권한 실패 증거

token이 없으면 401을 재시도하지 않고 실행을 중단한다.

```
result_file = "output/course-labs/day4/02_auth_status.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

.env.sample이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

GitHub 인증과 최소 권한 정상 Case

NORMAL

.env는 ignore되고 .env.sample만 추적된다.

- 01 — .env.sample 입력과 command가 기록됐는가
- 02 — .env.sample schema가 validate되는가
- 03 — GitHub 인증과 최소 권한 focused test가 실제로 통과했는가

GitHub 인증과 최소 권한 경계 Case

BOUNDARY

token이 없으면 401을 재시도하지 않고 실행을 중단한다.

- 01 — GitHub 인증과 최소 권한: raw traceback 대신 stable error_code가 남는가
- 02 — .env.sample: 외부 쓰기·자동 메일이 false인가
- 03 — token이 없으면 401을 재시도하지 않고 실행을 중단한다. 재현이 가능한가

GitHub 인증과 최소 권한 현업 적용

교육에서는 mock, 운영에서는 App token·short-lived credential 고려

- 01 — GitHub 인증과 최소 권한은 합성·비식별 sample로 먼저 실행
- 02 — 교육에서는 mock, 운영에서는 App token·short-lived credential 고려의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — .env.sample 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — .env.sample을 운영 검토 자료로 사용

GitHub 인증과 최소 권한 포트폴리오 적용

GitHub 인증과 최소 권한 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — token 값을 notebook output이나 screenshot에 노출한다.의 사용자 영향을 한 문장으로 설명
- 02 — GitHub 인증과 최소 권한 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — secret을 읽거나 출력하지 않는 범위에서 config validation만 구현한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — .env.sample과 README 재현 절차를 제출

GitHub 인증과 최소 권한 운영 점검

품질

무엇을 .env는 ignore되고 .env.sample만 추적된다.로 정의하는가

복구

token 값을 notebook output이나 screenshot에 노출한다. 발생 시 .env.sample에서 원인 상태를 확인

안전

GitHub 인증과 최소 권한의 사람 승인과 external_write=false 위치

관측

.env.sample에서 어떤 status_error_code를 보는가

.env.sample 실행 증거

- 01 — 설명: GitHub 인증과 최소 권한이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `git check-ignore -v .env`
- 03 — 증거: .env.sample과 정상·실패 test 결과가 남아 있다.

다음 차시: REST 상태 코드와 복구

DAY 4 · 3차시

10:40-11:30 (쉬는 시간 11:30-12:00)

REST 상태 코드와 복구

REST · Pagination · github_read_result.json

진행

REST 상태 코드와 복구
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/github_service.py
materials/day4/
day4_service_lab.ipynb

확인할 결과

github_read_result.json

3차시 입력과 결과

입력	———	.env.sample
이번 차시	———	REST 상태 코드와 복구
결과	———	github_read_result.json
다음 연결	———	LangGraph State와 분기

REST 상태 코드와 복구 필요성

모든 오류를 같은 `retry`로 처리하면 인증 문제와 `rate limit`이 장애를 키운다.

GET request에서 `github_read_result.json`까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: `github_read_result.json`

REST 상태 코드와 복구 학습 결과

- 01 — read-only PR 조회와 pagination·rate limit 계약을 만든다.
- 02 — 401·403·404·422·429를 모두 세 번 retry한다. 왜 위험한지 설명한다.
- 03 — github_read_result.json에서 정상과 실패 상태를 직접 확인한다.

REST 상태 코드와 복구 용어

용어	이 수업에서 쓰는 뜻
REST	HTTP resource 단위 API 방식
Pagination	여러 페이지로 결과를 나눔
Rate limit	시간당 요청 제한
Retry-After	다시 요청 가능한 시각 힌트

github_read_result.json 입출력

INPUT

src/course_services/github_service.py

OUTPUT

github_read_result.json

PROCESS

GET request → status classify → pagination

VERIFY

200 응답은 target·diff·rate metadata를 반환한다.
429만 Retry-After와 최대 횟수 안에서 재시도한다.

REST 상태 코드와 복구 실행 흐름

01

GET request

02

status classify

03

pagination

04

bounded backoff

05

structured result

마지막 판단은 `github_read_result.json`에서 사람이 확인합니다.

github_read_result.json 정상·실패 계약

정상	——	200 응답은 target·diff·rate metadata를 반환한다.
경계	——	429만 Retry-After와 최대 횟수 안에서 재시도한다.
외부 쓰기	——	REST 상태 코드와 복구: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·github_read_result.json

REST 상태 코드와 복구 파일 지도

입력·fixture

src/course_services/github_service.py

구현 코드

materials/day4/day4_service_lab.ipynb

검증·test

AGENTS.md

따라하기 notebook

output/day4-github/read-result.json

REST 상태 코드와 복구 개념·코드

REST

HTTP resource 단위 API 방식

```
src/course_services/github_service.py
```

Rate limit

시간당 요청 제한

```
AGENTS.md
```

Pagination

여러 페이지로 결과를 나눔

```
materials/day4/day4_service_lab.ipynb
```

Retry-After

다시 요청 가능한 시각 힌트

```
output/day4-github/read-result.json
```

REST 상태 코드와 복구 선택 기준

구분	역할	이번 차시의 판단 기준
REST	GET request 단계의 입력 기준	결정론 test로 먼저 확인
Pagination	status classify 단계의 교체 경계	adapter 경계를 확인
Rate limit	pagination 단계의 운영 조건	비용·지연·권한을 확인
Retry-After	bounded backoff 단계의 판단 기록	사람 판단과 운영 기록을 확인

REST 상태 코드와 복구 대표 실패

실패

401·403·404·422·429를 모두 세 번 retry한다.

사용자 영향

401·403·404·422·429를 모두 세 번 retry한다.
상태에서는 github_read_result.json을 운영
판단에 사용할 수 없다.

REST 상태 코드와 복구 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

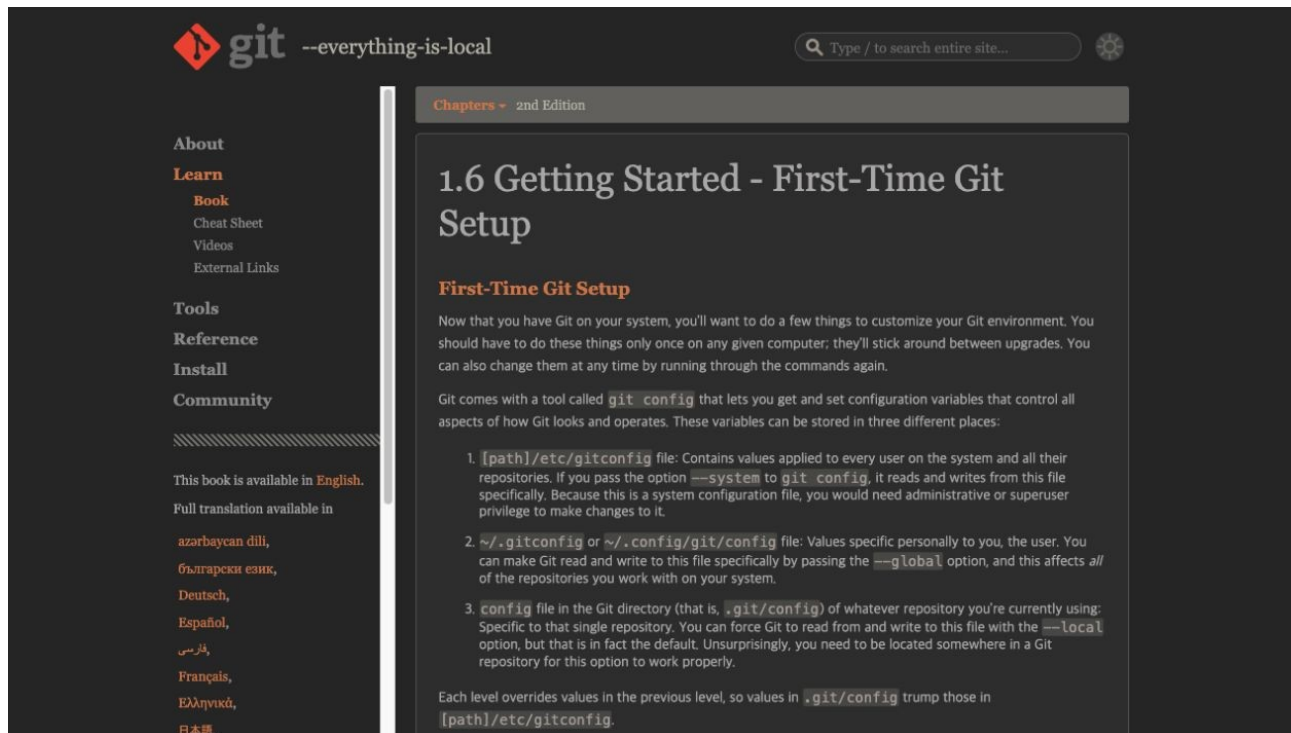
REST 상태 코드와 복구 복구 경로

- 01 — 탐지: 429만 Retry-After와 최대 횟수 안에서 재시도한다.
- 02 — 중단: 401·403·404·422·429를 모두 세 번 retry한다.
- 03 — 복구: 재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.
- 04 — 증거: github_read_result.json과 test 결과를 함께 남긴다.

REST 상태 코드와 복구 Codex 검증 루프

materials/day4/
day4_service_lab.ipynb의
책임을 찾고
github_read_result.json 계약을
focused test로 고정합니다.

REST 상태 코드와 복구
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



REST 상태 코드와 복구 Codex 작업 명세

목표

fake HTTP response로 각 status code의 결정론 test를 작성한다.

허용 경로

materials/day4/day4_service_lab.ipynb
AGENTS.md

완료 조건

200 응답은 target·diff·rate metadata를 반환한다.
429만 Retry-After와 최대 횟수 안에서 재시도한다.

금지

REST 상태 코드와 복구: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

REST 상태 코드와 복구 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | REST 상태 코드와 복구 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 200 응답은 target·diff·rate metadata를 반환한다. |
| 03 | Claude review | —— | github_read_result.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | REST 상태 코드와 복구의 두 LLM 의견과 test는 자동 승인이 아니다 |

REST 상태 코드와 복구 정상 시연

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `materials/day4/day4_service_lab.ipynb`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py`
- 04 — 성공 신호: 200 응답은 `target·diff·rate metadata`를 반환한다.

REST 상태 코드와 복구 실행 명령

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

REST 상태 코드와 복구: repository root · Python 3.12 · .env 미출력

03_github_response_contracts.json 실행 결과

```
{
  "200": {
    "status": "SUCCESS",
    "status_code": 200,
    "action": "USE_RESPONSE",
    "retryable": false,
    "retry_after_seconds": null,
    "error_code": null
  },
  "401": {
    "status": "EXPECTED_FAILURE",
    "status_code": 401,
    "action": "STOP_AND_REAUTHENTICATE",
    "retryable": false,
    "retry_after_seconds": null,
    "error_code": "GITHUB_AUTH_REQUIRED"
  },
  "403": {
    "status": "EXPECTED_FAILURE",
    "status_code": 403,
    "action": "STOP_AND_CHECK_PERMISSION",
    "retryable": false,
    "retry_after_seconds": null,
    "error_code": "GITHUB_PERMISSION_DENIED"
  },
  ...
}
```

REST 상태 코드와 복구 실패·복구 시연

재현

429만 Retry-After와 최대 횟수 안에서
재시도한다.

복구

재인증·권한 확인·target 확인·diff 갱신·
대기 경로를 error code별로 나눈다.

확인: 429만 Retry-After와 최대 횟수 안에서 재시도한다.

REST 상태 코드와 복구 소프트웨어 제작

열 파일 `materials/day4/day4_service_lab.ipynb`
`AGENTS.md`

작업 범위 **REST 상태 코드와 복구 명세 → 최소 Patch → Focused Test → Diff 검토**

완료 기준 **200 응답은 target-diff-rate metadata를 반환한다.**
`github_read_result.json` 저장

REST 상태 코드와 복구 Notebook 셀

materials/day4/day4_service_lab.ipynb · 3차시

```
from src.course_services.github_service import classify_github_response

response_contracts = {
    str(status): classify_github_response(status, retry_after_seconds=3)
    for status in (200, 401, 403, 404, 422, 429, 500)
}
assert response_contracts["429"]["retryable"] is True
assert all(not response_contracts[str(status)]["retryable"] for status in (401, 403, 404, 422, 500))
save_json("03_github_response_contracts.json", response_contracts)
response_contracts
```

REST 상태 코드와 복구 Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py
```

저장 위치

```
output/course-labs/day4/03_github_response_contracts.json
```

200 응답은 target·diff·rate metadata를 반환한다.
실패 시 github_read_result.json의 status·error_code를 먼저 확인

REST 상태 코드와 복구 실패 증거

429만 Retry-After와 최대 횟수 안에서 재시도한다.

```
result_file = "output/course-labs/day4/03_github_response_contracts.json"
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"
external_write = false
```

github_read_result.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

REST 상태 코드와 복구 정상 Case

NORMAL

200 응답은 target·diff·rate metadata를 반환한다.

- 01 — src/course_services/github_service.py 입력과 command가 기록됐는가
- 02 — github_read_result.json schema가 validate되는가
- 03 — REST 상태 코드와 복구 focused test가 실제로 통과했는가

REST 상태 코드와 복구 경계 Case

BOUNDARY

429만 Retry-After와 최대 횟수 안에서 재시도한다.

- 01 — REST 상태 코드와 복구: raw traceback 대신 stable error_code가 남는가
- 02 — github_read_result.json: 외부 쓰기·자동 메일이 false인가
- 03 — 429만 Retry-After와 최대 횟수 안에서 재시도한다. 재현이 가능한가

REST 상태 코드와 복구 현업 적용

PR 정보를 안전하게 수집하는 ingestion adapter

- 01 — REST 상태 코드와 복구는 합성·비식별 sample로 먼저 실행
- 02 — PR 정보를 안전하게 수집하는 ingestion adapter의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — github_read_result.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — github_read_result.json을 운영 검토 자료로 사용

REST 상태 코드와 복구 포트폴리오 적용

REST 상태 코드와 복구 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 401·403·404·422·429를 모두 세 번 retry한다.의 사용자 영향을 한 문장으로 설명
- 02 — REST 상태 코드와 복구 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — fake HTTP response로 각 status code의 결정론 test를 작성한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — github_read_result.json과 README 재현 절차를 제출

REST 상태 코드와 복구 운영 점검

품질

무엇을 200 응답은 target·diff·rate metadata를 반환한다.로 정의하는가

복구

401·403·404·422·429를 모두 세 번 retry한다. 발생 시 github_read_result.json에서 원인 상태를 확인

안전

REST 상태 코드와 복구의 사람 승인과 external_write=false 위치

관측

github_read_result.json에서 어떤 status·error_code를 보는가

github_read_result.json 실행 증거

- 01 — 설명: REST 상태 코드와 복구가 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: github_read_result.json과 정상·실패 test 결과가 남아 있다.

다음 차시: LangGraph State와 분기

DAY 4 · 4차시

12:00-12:50 (쉬는 시간 13:40-14:00)

LangGraph State와 분기

State · Node · compiled_review_graph

진행

LangGraph State와 분기
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/langgraph_lab.py
src/meeting_agent_workflow.py

확인할 결과

compiled_review_graph

4차시 입력과 결과

입력	——	github_read_result.json
이번 차시	——	LangGraph State와 분기
결과	——	compiled_review_graph
다음 연결	——	Checkpoint와 Idempotency

LangGraph State와 분기 필요성

모든 원문과 client 객체를 state에 넣으면 checkpoint와 재시작이 불안정해진다.

validate target에서 compiled_review_graph까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: `compiled_review_graph`

LangGraph State와 분기 학습 결과

- 01 — PR review의 node·edge·conditional edge를 조립한다.
- 02 — state에 token·client·raw audio를 넣어 checkpoint에 남긴다. 왜 위험한지 설명한다.
- 03 — compiled_review_graph에서 정상과 실패 상태를 직접 확인한다.

LangGraph State와 분기 용어

용어	이 수업에서 쓰는 뜻
State	node 사이에 전달할 명시 데이터
Node	한 책임을 실행하는 함수
Edge	다음 실행 순서
Conditional edge	상태에 따른 분기

compiled_review_graph 입출력

INPUT

src/langgraph_lab.py

OUTPUT

compiled_review_graph

PROCESS

validate target → load diff → review

VERIFY

approve·edit·reject가 서로 다른 terminal state로 간다.
validation error는 publish node에 도달하지 않는다.

LangGraph State와 분기 실행 흐름

01

validate target

02

load diff

03

review

04

human review

05

dry-run

마지막 판단은 `compiled_review_graph`에서 사람이 확인합니다.

compiled_review_graph 정상·실패 계약

정상	approve·edit·reject가 서로 다른 terminal state로 간다.
경계	validation error는 publish node에 도달하지 않는다.
외부 쓰기	LangGraph State와 분기: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	status·error_code·입력 식별자·compiled_review_graph

LangGraph State와 분기 파일 지도

입력·fixture

src/langgraph_lab.py

구현 코드

src/meeting_agent_workflow.py

검증·test

src/course_services/github_service.py

따라하기 notebook

materials/day4/day4_service_lab.ipynb

LangGraph State와 분기 개념·코드

State

node 사이에 전달할 명시 데이터

```
src/langgraph_lab.py
```

Edge

다음 실행 순서

```
src/course_services/github_service.py
```

Node

한 책임을 실행하는 함수

```
src/meeting_agent_workflow.py
```

Conditional edge

상태에 따른 분기

```
materials/day4/day4_service_lab.ipynb
```

LangGraph State와 분기 선택 기준

구분	역할	이번 차시의 판단 기준
State	validate target 단계의 입력 기준	결정론 test로 먼저 확인
Node	load diff 단계의 교체 경계	adapter 경계를 확인
Edge	review 단계의 운영 조건	비용·지연·권한을 확인
Conditional edge	human review 단계의 판단 기록	사람 판단과 운영 기록을 확인

LangGraph State와 분기 대표 실패

실패

state에 token·client·raw audio를
넣어 checkpoint에 남긴다.

사용자 영향

state에 token·client·raw audio를 넣어
checkpoint에 남긴다. 상태에서는
compiled_review_graph을 운영 판단에 사용할
수 없다.

LangGraph State와 분기 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

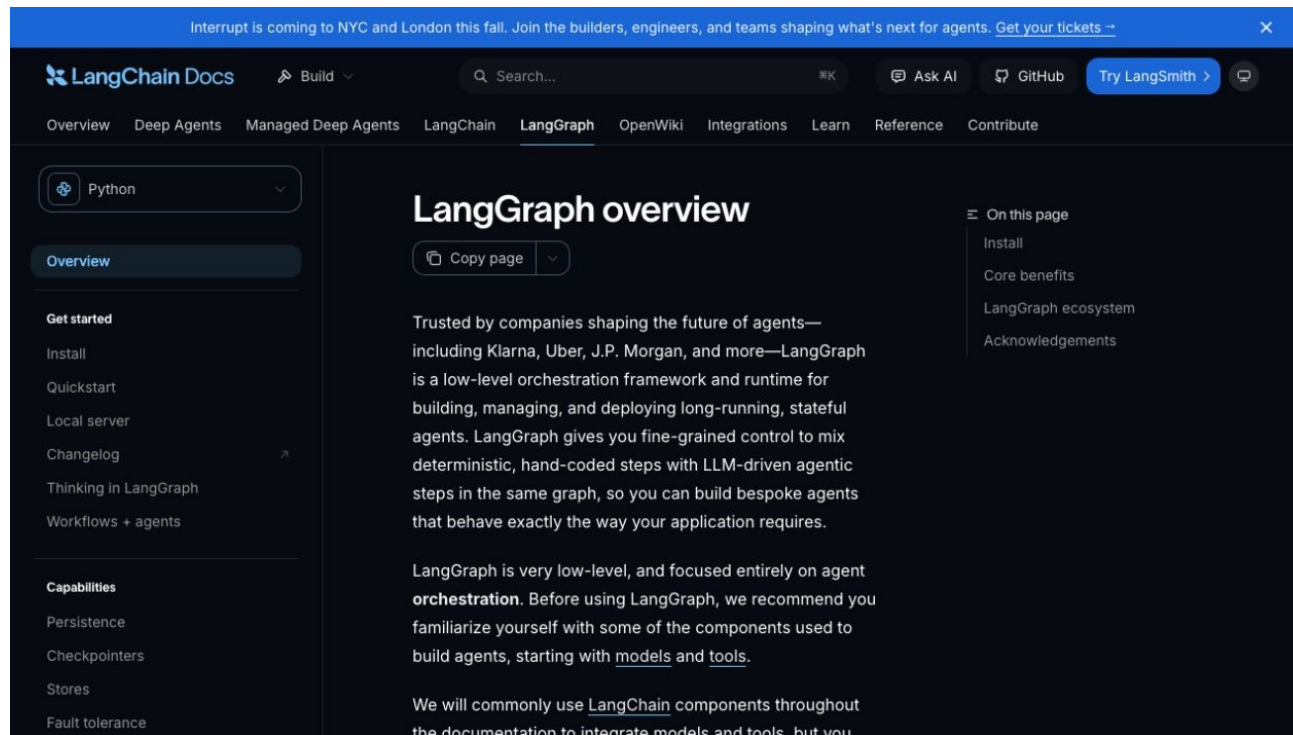
LangGraph State와 분기 복구 경로

- 01 — 탐지: validation error는 publish node에 도달하지 않는다.
- 02 — 중단: state에 token·client·raw audio를 넣어 checkpoint에 남긴다.
- 03 — 복구: ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서 주입한다.
- 04 — 증거: compiled_review_graph과 test 결과를 함께 남긴다.

LangGraph State와 분기 Codex 검증 루프

src/
meeting_agent_workflow.py의
책임을 찾고
compiled_review_graph 계약을
focused test로 고정합니다.

LangGraph State와 분기
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



LangGraph State와 분기 Codex 작업 명세

목표

기존 **service** 함수를 **node**로 감싸되 **contract**를 바꾸지 않게 한다.

허용 경로

src/meeting_agent_workflow.py
src/course_services/github_service.py

완료 조건

approve·edit·reject가 서로 다른 terminal state로 간다.
validation error는 publish node에 도달하지 않는다.

금지

LangGraph State와 분기: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

LangGraph State와 분기 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | LangGraph State와 분기 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | approve·edit·reject가 서로 다른 terminal state로 간다. |
| 03 | Claude review | —— | compiled_review_graph 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | LangGraph State와 분기의 두 LLM 의견과 test는 자동 승인이 아니다 |

LangGraph State와 분기 정상 시연

- 01 — 열기: `src/langgraph_lab.py`
- 02 — 구현 확인: `src/meeting_agent_workflow.py`
- 03 — 실행: `python -m src.langgraph_lab --decision all`
- 04 — 성공 신호: `approve·edit·reject`가 서로 다른 terminal state로 간다.

LangGraph State와 분기 실행 명령

```
$ python -m src.langgraph_lab --decision all
```

실행 전 확인

LangGraph State와 분기: repository root · Python 3.12 · .env 미출력

04_graph_states.json 실행 결과

```
{
  "approve": {
    "status": "READY_FOR_EXPORT",
    "framework": "LangGraph",
    "request_id": "day4-approve",
    "thread_id_reused": true,
    "interrupted": true,
    "interrupt_payload": {
      "question": "회의 결과를 승인·수정·거절하시겠습니까?",
      "request_id": "day4-approve",
      "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.",
      "action_items": [
        {
          "task": "공개 FAQ 30건 비식별 샘플 정리",
          "owner": "서연",
          "due_date": "2026-08-27",
          "evidence_ids": [
            "s12",
            "s18"
          ]
        }
      ],
    },
    {
      "task": "응답 JSON Schema와 실패 테스트 작성",
      "owner": "준호",
      "due_date": "2026-08-28",
    }
  },
  ...
}
```

LangGraph State와 분기 실패·복구 시연

재현

validation error는 publish node에 도달하지 않는다.

복구

ID·status·검증 결과만 state에 두고 외부
자원은 adapter 경계에서 주입한다.

확인: validation error는 publish node에 도달하지 않는다.

LangGraph State와 분기 소프트웨어 제작

열 파일

`src/meeting_agent_workflow.py`
`src/course_services/github_service.py`

작업 범위

LangGraph State와 분기 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

`approve·edit·reject`가 서로 다른 `terminal state`로 간다.
`compiled_review_graph` 저장

LangGraph State와 분기 Notebook 셀

materials/day4/day4_service_lab.ipynb · 4차시

```
from src.langchain_lab import fixture_payload
from src.langgraph_lab import run_langgraph_lab

draft = fixture_payload()
graph_results = {
    decision: run_langgraph_lab(draft, decision=decision, request_id=f"day4-{decision}")
    for decision in ("approve", "edit", "reject")
}
terminal_states = {key: value["final_state"]["status"] for key, value in graph_results.items()}
assert terminal_states["reject"] == "REJECTED"
save_json("04_graph_states.json", graph_results)
terminal_states
```


LangGraph State와 분기 Terminal 실행

```
$ python -m src.langgraph_lab --decision all
```

저장 위치

```
output/course-labs/day4/04_graph_states.json
```

approve·edit·reject가 서로 다른 terminal state로 간다.
실패 시 compiled_review_graph의 status·error_code를 먼저 확인

LangGraph State와 분기 실패 증거

validation error는 publish node에 도달하지 않는다.

```
result_file = "output/course-labs/day4/04_graph_states.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

compiled_review_graph이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

LangGraph State와 분기 정상 Case

NORMAL

approve·edit·reject가 서로 다른 terminal state로 간다.

- 01 — src/langgraph_lab.py 입력과 command가 기록됐는가
- 02 — compiled_review_graph schema가 validate되는가
- 03 — LangGraph State와 분기 focused test가 실제로 통과했는가

LangGraph State와 분기 경계 Case

BOUNDARY

validation error는 publish node에 도달하지 않는다.

- 01 — LangGraph State와 분기: raw traceback 대신 stable error_code가 남는가
- 02 — compiled_review_graph: 외부 쓰기·자동 메일이 false인가
- 03 — validation error는 publish node에 도달하지 않는다. 재현이 가능한가

LangGraph State와 분기 현업 적용

회의 Agent와 PR Agent가 같은 승인 패턴을 재사용

- 01 — LangGraph State와 분기는 합성·비식별 sample로 먼저 실행
- 02 — 회의 Agent와 PR Agent가 같은 승인 패턴을 재사용의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — compiled_review_graph 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — compiled_review_graph을 운영 검토 자료로 사용

LangGraph State와 분기 포트폴리오 적용

LangGraph State와 분기 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — state에 token·client·raw audio를 넣어 checkpoint에 남긴다.의 사용자 영향을 한 문장으로 설명
- 02 — LangGraph State와 분기 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 기존 service 함수를 node로 감싸되 contract를 바꾸지 않게 한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — compiled_review_graph과 README 재현 절차를 제출

LangGraph State와 분기 운영 점검

품질

무엇을 approve·edit·reject가 서로 다른 terminal state로 간다.로 정의하는가

복구

state에 token·client·raw audio를 넣어 checkpoint에 남긴다. 발생 시 compiled_review_graph에서 원인 상태를 확인

안전

LangGraph State와 분기의 사람 승인과 external_write=false 위치

관측

compiled_review_graph에서 어떤 status·error_code를 보는가

compiled_review_graph 실행 증거

- 01 — 설명: LangGraph State와 분기이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langgraph_lab --decision all`
- 03 — 증거: `compiled_review_graph`과 정상·실패 test 결과가 남아 있다.

다음 차시: Checkpoint와 Idempotency

12:50-13:40 (쉬는 시간 13:40-14:00)

Checkpoint와 Idempotency

Checkpoint · Thread ID · checkpoint_and_audit.json

진행

Checkpoint와 Idempotency
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/github_service.py
src/langgraph_lab.py

확인할 결과

checkpoint_and_audit.json

5차시 입력과 결과

입력	————	compiled_review_graph
이번 차시	————	Checkpoint와 Idempotency
결과	————	checkpoint_and_audit.json
다음 연결	————	Human Approval과 Interrupt

Checkpoint와 Idempotency 필요성

interrupt 전후 node가 다시 실행돼도 외부 쓰기는 한 번이어야 한다.

request ID에서 checkpoint_and_audit.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: **checkpoint_and_audit.json**

Checkpoint와 Idempotency 학습 결과

- 01 — retry·checkpoint·request ID·idempotency를 연결한다.
- 02 — timeout 뒤 동일 comment를 다시 게시한다. 왜 위험한지 설명한다.
- 03 — checkpoint_and_audit.json에서 정상과 실패 상태를 직접 확인한다.

Checkpoint와 Idempotency 용어

용어	이 수업에서 쓰는 뜻
Checkpoint	중단 시 state 저장
Thread ID	같은 실행을 이어 가는 식별자
Idempotency	같은 요청을 반복해도 결과가 한 번인 성질
Backoff	재시도 간격을 점차 늘림

checkpoint_and_audit.json 입출력

INPUT

src/course_services/github_service.py

PROCESS

request ID → checkpoint → bounded retry

OUTPUT

checkpoint_and_audit.json

VERIFY

첫 실행은 reused=false, 두 번째는 true다.
body나 head SHA가 달라지면 새 승인 대상으로 본다.

Checkpoint와 Idempotency 실행 흐름

01

request ID

02

checkpoint

03

bounded retry

04

idempotency
lookup

05

reuse result

마지막 판단은 `checkpoint_and_audit.json`에서 사람이 확인합니다.

checkpoint_and_audit.json 정상·실패 계약

정상	——	첫 실행은 reused=false, 두 번째는 true다.
경계	——	body나 head SHA가 달라지면 새 승인 대상으로 본다.
외부 쓰기	——	Checkpoint와 Idempotency: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·checkpoint_and_audit.json

Checkpoint와 Idempotency 파일 지도

입력·fixture

`src/course_services/github_service.py`

구현 코드

`src/langgraph_lab.py`

검증·test

`tests/test_course_services.py`

따라하기 notebook

`materials/day4/day4_service_lab.ipynb`

Checkpoint와 Idempotency 개념·코드

Checkpoint

중단 시 state 저장

```
src/course_services/github_service.py
```

Thread ID

같은 실행을 이어 가는 식별자

```
src/langgraph_lab.py
```

Idempotency

같은 요청을 반복해도 결과가 한 번인 성질

```
tests/test_course_services.py
```

Backoff

재시도 간격을 점차 늘림

```
materials/day4/day4_service_lab.ipynb
```

Checkpoint와 Idempotency 선택 기준

구분	역할	이번 차시의 판단 기준
Checkpoint	request ID 단계의 입력 기준	결정론 test로 먼저 확인
Thread ID	checkpoint 단계의 교체 경계	adapter 경계를 확인
Idempotency	bounded retry 단계의 운영 조건	비용·지연·권한을 확인
Backoff	idempotency lookup 단계의 판단 기록	사람 판단과 운영 기록을 확인

Checkpoint와 Idempotency 대표 실패

실패

timeout 뒤 동일 comment를 다시
게시한다.

사용자 영향

timeout 뒤 동일 comment를 다시 게시한다.
상태에서는 checkpoint_and_audit.json을
운영 판단에 사용할 수 없다.

Checkpoint와 Idempotency 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

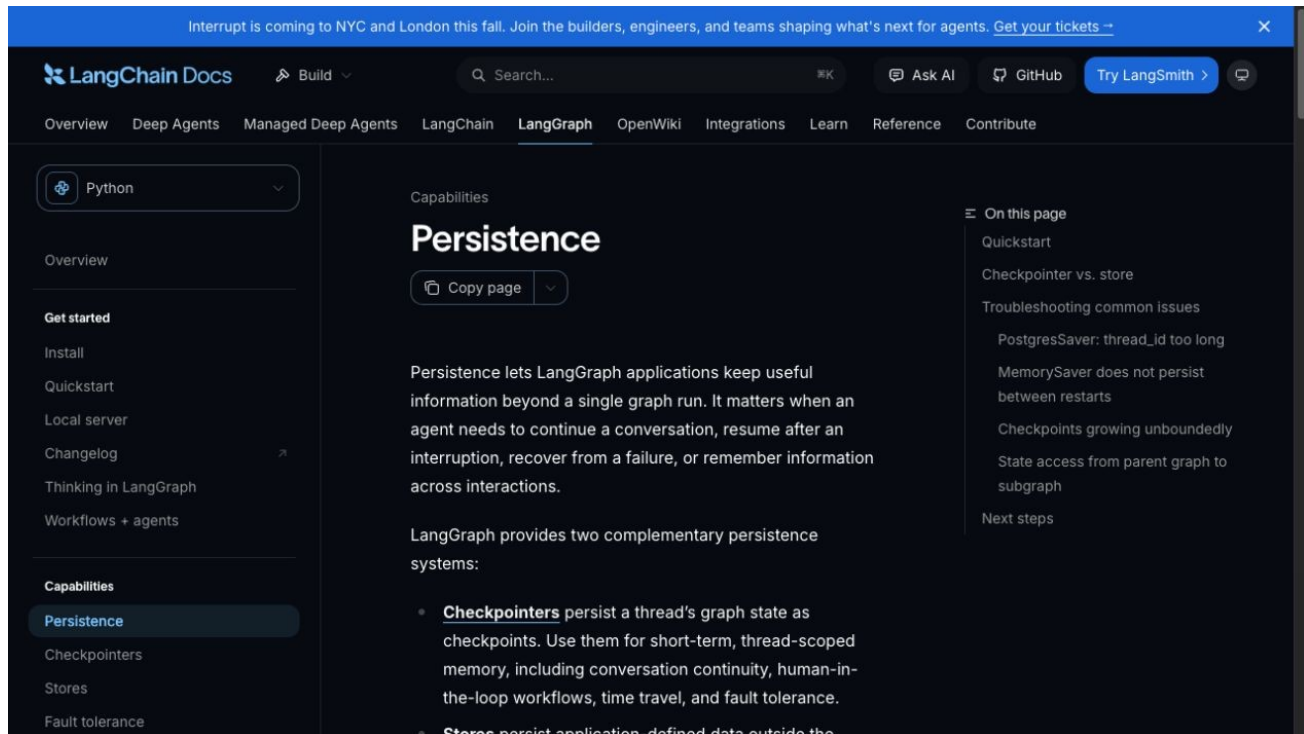
Checkpoint와 Idempotency 복구 경로

- 01 — 탐지: body나 head SHA가 달라지면 새 승인 대상으로 본다.
- 02 — 중단: timeout 뒤 동일 comment를 다시 게시한다.
- 03 — 복구: target·SHA·body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.
- 04 — 증거: checkpoint_and_audit.json과 test 결과를 함께 남긴다.

Checkpoint와 Idempotency Codex 검증 루프

src/langgraph_lab.py의 책임을
찾고
checkpoint_and_audit.json
계약을 focused test로 고정합니다.

Checkpoint와 Idempotency
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



Checkpoint와 Idempotency Codex 작업 명세

목표

같은 plan을 두 번 실행해 publisher call이 1회인지 test한다.

허용 경로

src/langgraph_lab.py
tests/test_course_services.py

완료 조건

첫 실행은 reused=false, 두 번째는 true다.
body나 head SHA가 달라지면 새 승인 대상으로 본다.

금지

Checkpoint와 Idempotency: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Checkpoint와 Idempotency 독립 리뷰

- | | | | |
|----|----------------------|------|--|
| 01 | Codex diff | ———— | Checkpoint와 Idempotency 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | ———— | 첫 실행은 reused=false, 두 번째는 true다. |
| 03 | Claude review | ———— | checkpoint_and_audit.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | ———— | Checkpoint와 Idempotency의 두 LLM 의견과 test는 자동 승인이 아니다 |

Checkpoint와 Idempotency 정상 시연

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `src/langgraph_lab.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k idempotent`
- 04 — 성공 신호: 첫 실행은 `reused=false`, 두 번째는 `true`다.

Checkpoint와 Idempotency 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k idempotent
```

실행 전 확인

Checkpoint와 Idempotency: repository root · Python 3.12 · .env 미출력

05_idempotency.json 실행 결과

```
{
  "first": {
    "status": "PUBLISHED",
    "target": {
      "repository": "classroom-owner/agent-sandbox",
      "number": 17,
      "head_sha": "7c83e8fce9a24d90b6d4a6c9123aa90817b65b1a"
    },
    "body": "## 교육용 코드 리뷰 초안\n\n- **P0 · 검증되지 않은 문자열이 코드로 실행될 수 있음**\n  - 위치: `src/payment_job.py:9`\n  - 근거: `result = eval(command)`\n  - 제안: 허용된 명령어나 파서만 호출하는 명시적 registry로 교체하세요.\n- **P1 · 외부 쓰기 전에 사람 승인 경계가 보이지 않음**\n  - 위치: `src/payment_job.py:11`\n  - 근거: `requests.post(\"https://example.invalid/jobs\", json=payload)`\n  - 제안: dry-run payload, 대상 확인, human_approved 조건을 쓰기 호출 앞에 두세요.\n- **P2 · 넓은 예외 처리가 실패 계약을 지움**\n  - 위치: `src/payment_job.py:12`\n  - 근거: `except Exception:`\n  - 제안: 복구 가능한 예외만 잡고 구조화된 error_code를 반환하세요.\n\n자동 게시하지 않았습니다. 대상과 내용을 사람이 확인해야 합니다.",
    "idempotency_key": "c97ba39c959d7fda585b86c3",
    "human_approval_required": true,
    "external_write": true,
    "error_code": null,
    "remote_result": {
      "id": 101,
      "url": "https://example.invalid/reviews/101",
      "reused": false
    }
  },
  "second": {
    "status": "PUBLISHED",
    "target": {
      "repository": "classroom-owner/agent-sandbox",
      "number": 17,
      "head_sha": "7c83e8fce9a24d90b6d4a6c9123aa90817b65b1a"
    }
  },
  ...
}
```

Checkpoint와 Idempotency 실패·복구 시연

재현

body나 head SHA가 달라지면 새 승인 대상으로 본다.

복구

target·SHA·body hash를
idempotency key로 저장하고 기존
remote result를 재사용한다.

확인: body나 head SHA가 달라지면 새 승인 대상으로 본다.

Checkpoint와 Idempotency 소프트웨어 제작

열 파일	<code>src/langgraph_lab.py</code> <code>tests/test_course_services.py</code>
작업 범위	Checkpoint와 Idempotency 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	첫 실행은 <code>reused=false</code> , 두 번째는 <code>true</code> 다. <code>checkpoint_and_audit.json</code> 저장

Checkpoint와 Idempotency Notebook 셀

materials/day4/day4_service_lab.ipynb · 5차시

```
from src.course_services.github_service import InMemoryIdempotencyStore, prepare_review_comment, publish_review_comment
from src.course_services.review_service import run_review_service

diff_text = (ROOT / fixture["diff"]).read_text(encoding="utf-8")
report = run_review_service(diff_text)
approval_plan = prepare_review_comment(report=report, target=target, dry_run=False)
store = InMemoryIdempotencyStore()
calls = []
def fake_publisher(pr_target, body):
    calls.append({"target": pr_target.model_dump(), "body_length": len(body)})
    return {"id": 101, "url": "https://example.invalid/reviews/101"}
first = publish_review_comment(plan=approval_plan, human_approved=True, publisher=fake_publisher, store=store)
second = publish_review_comment(plan=approval_plan, human_approved=True, publisher=fake_publisher, store=store)
idempotency_result = {"first": first, "second": second, "call_count": len(calls)}
assert len(calls) == 1 and second["remote_result"]["reused"] is True
save_json("05_idempotency.json", idempotency_result)
{"call_count": len(calls), "duplicate_reused": second["remote_result"]["reused"]}
```

Checkpoint와 Idempotency Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k idempotent
```

저장 위치

```
output/course-labs/day4/05_idempotency.json
```

첫 실행은 reused=false, 두 번째는 true다.
실패 시 checkpoint_and_audit.json의 status·error_code를 먼저 확인

Checkpoint와 Idempotency 실패 증거

body나 head SHA가 달라지면 새 승인 대상으로 본다.

```
result_file = "output/course-labs/day4/05_idempotency.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

checkpoint_and_audit.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Checkpoint와 Idempotency 정상 Case

NORMAL

첫 실행은 reused=false, 두 번째는 true다.

- 01 — src/course_services/github_service.py 입력과 command가 기록됐는가
- 02 — checkpoint_and_audit.json schema가 validate되는가
- 03 — Checkpoint와 Idempotency focused test가 실제로 통과했는가

Checkpoint와 Idempotency 경계 Case

BOUNDARY

body나 head SHA가 달라지면 새 승인 대상으로 본다.

- 01 — Checkpoint와 Idempotency: raw traceback 대신 stable error_code가 남는가
- 02 — checkpoint_and_audit.json: 외부 쓰기·자동 메일이 false인가
- 03 — body나 head SHA가 달라지면 새 승인 대상으로 본다. 재현이 가능한가

Checkpoint와 Idempotency 현업 적용

네트워크가 불안한 CI bot의 중복 comment 방지

- 01 — Checkpoint와 Idempotency은 합성·비식별 sample로 먼저 실행
- 02 — 네트워크가 불안한 CI bot의 중복 comment 방지의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — checkpoint_and_audit.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — checkpoint_and_audit.json을 운영 검토 자료로 사용

Checkpoint와 Idempotency 포트폴리오 적용

Checkpoint와 Idempotency · 문제·코드·실패·검증을 한 저장소로 제출

- 01 ——— timeout 뒤 동일 comment를 다시 게시한다.의 사용자 영향을 한 문장으로 설명
- 02 ——— Checkpoint와 Idempotency 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 ——— 같은 plan을 두 번 실행해 publisher call이 1회인지 test한다.의 diff와 직접 검토한 test 증거를 구분
- 04 ——— checkpoint_and_audit.json과 README 재현 절차를 제출

Checkpoint와 Idempotency 운영 점검

품질

무엇을 첫 실행은 reused=false, 두 번째는 true다.로 정의하는가

복구

timeout 뒤 동일 comment를 다시 게시한다. 발생 시 checkpoint_and_audit.json에서 원인 상태를 확인

안전

Checkpoint와 Idempotency의 사람 승인과 external_write=false 위치

관측

checkpoint_and_audit.json에서 어떤 status·error_code를 보는가

checkpoint_and_audit.json 실행 증거

- 01 — 설명: Checkpoint와 Idempotency이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k idempotent`
- 03 — 증거: checkpoint_and_audit.json과 정상·실패 test 결과가 남아 있다.

다음 차시: Human Approval과 Interrupt

DAY 4 · 6차시

15:00-15:50 (쉬는 시간 17:30-18:00)

Human Approval과 Interrupt

Interrupt · Approve · review_decision.json

진행

Human Approval과 Interrupt
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/langgraph_lab.py
src/meeting_agent_workflow.py

확인할 결과

review_decision.json

6차시 입력과 결과

입력	————	checkpoint_and_audit.json
이번 차시	————	Human Approval과 Interrupt
결과	————	review_decision.json
다음 연결	————	Dry-run Comment Payload

Human Approval과 Interrupt 필요성

초안만 보여주고 대상과 근거를 숨기면 사람은 사실상 승인할 수 없다.

초안 생성에서 review_decision.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: review_decision.json

Human Approval과 Interrupt 학습 결과

- 01 — approve·edit·reject interrupt payload를 설계한다.
- 02 — interrupt를 일반 Exception으로 잡아 자동 진행한다. 왜 위험한지 설명한다.
- 03 — review_decision.json에서 정상과 실패 상태를 직접 확인한다.

Human Approval과 Interrupt 용어

용어	이 수업에서 쓰는 뜻
Interrupt	workflow를 멈추고 입력을 기다림
Approve	초안을 그대로 허용
Edit	사람이 수정 후 허용
Reject	외부 쓰기 없이 종료

review_decision.json 입출력

INPUT

src/langgraph_lab.py

OUTPUT

review_decision.json

PROCESS

초안 생성 → target 표시 → finding 근거

VERIFY

approve가 READY_FOR_EXPORT로 간다.
reject는 external_write=false와 REJECTED로 끝난다.

Human Approval과 Interrupt 실행 흐름

01

초안 생성

02

target 표시

03

finding 근거

04

사람 결정

05

같은 thread resume

마지막 판단은 review_decision.json에서 사람이 확인합니다.

review_decision.json 정상·실패 계약

정상	approve가 READY_FOR_EXPORT로 간다.
경계	reject는 external_write=false와 REJECTED로 끝난다.
외부 쓰기	Human Approval과 Interrupt: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	status·error_code·입력 식별자·review_decision.json

Human Approval과 Interrupt 파일 지도

입력·fixture

src/langgraph_lab.py

구현 코드

src/meeting_agent_workflow.py

검증·test

web-demo/app.js

따라하기 notebook

materials/day4/day4_service_lab.ipynb

Human Approval과 Interrupt 개념·코드

Interrupt

workflow를 멈추고 입력을 기다림

```
src/langgraph_lab.py
```

Approve

초안을 그대로 허용

```
src/meeting_agent_workflow.py
```

Edit

사람이 수정 후 허용

```
web-demo/app.js
```

Reject

외부 쓰기 없이 종료

```
materials/day4/day4_service_lab.ipynb
```


Human Approval과 Interrupt 선택 기준

구분	역할	이번 차시의 판단 기준
Interrupt	초안 생성 단계의 입력 기준	결정론 test로 먼저 확인
Approve	target 표시 단계의 교체 경계	adapter 경계를 확인
Edit	finding 근거 단계의 운영 조건	비용·지연·권한을 확인
Reject	사람 결정 단계의 판단 기록	사람 판단과 운영 기록을 확인

Human Approval과 Interrupt 대표 실패

실패

interrupt를 일반 Exception으로 잡아 자동 진행한다.

사용자 영향

interrupt를 일반 Exception으로 잡아 자동 진행한다. 상태에서는 review_decision.json을 운영 판단에 사용할 수 없다.

Human Approval과 Interrupt 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

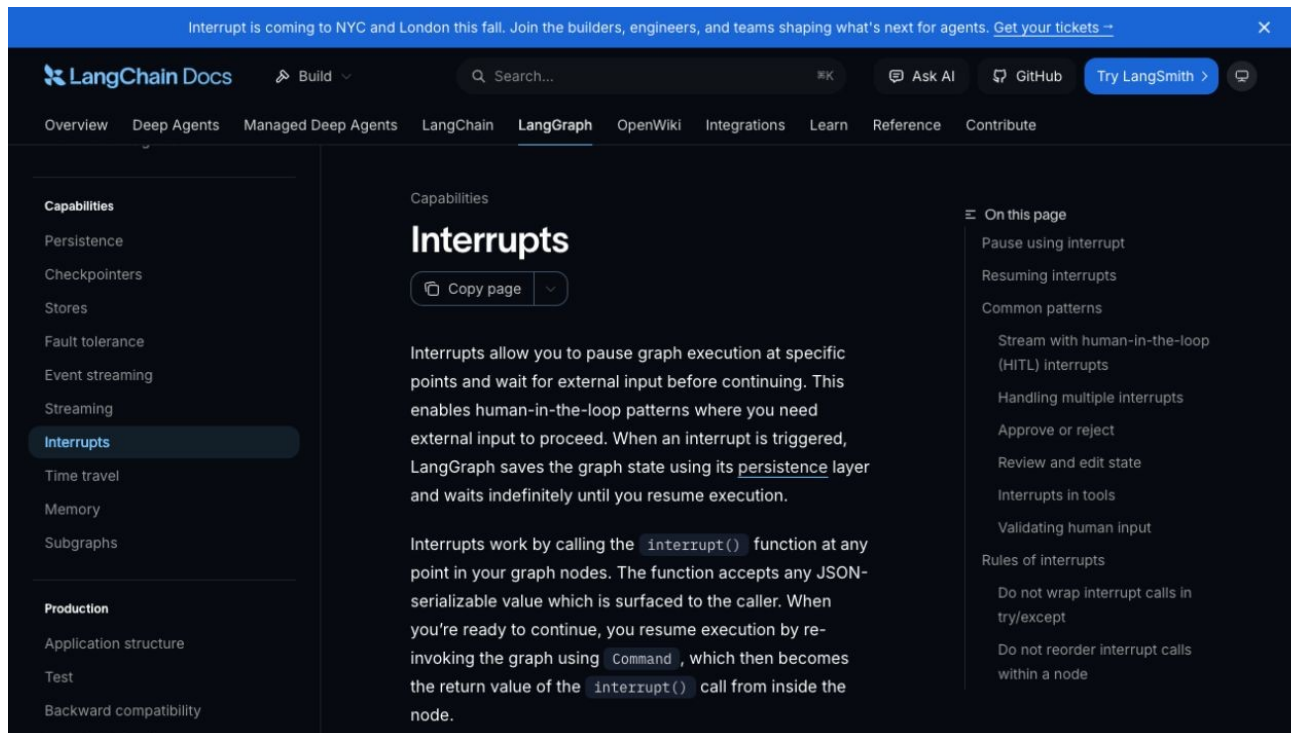
Human Approval과 Interrupt 복구 경로

- 01 — 탐지: reject는 external_write=false와 REJECTED로 끝난다.
- 02 — 중단: interrupt를 일반 Exception으로 잡아 자동 진행한다.
- 03 — 복구: interrupt를 workflow event로 두고 같은 thread_id와 JSON decision으로 resume한다.
- 04 — 증거: review_decision.json과 test 결과를 함께 남긴다.

Human Approval과 Interrupt Codex 검증 루프

src/
meeting_agent_workflow.py의
책임을 찾고
review_decision.json 계약을
focused test로 고정합니다.

Human Approval과 Interrupt
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



Human Approval과 Interrupt Codex 작업 명세

목표

승인 node 전에 외부 쓰기 호출이 존재하지 않는지 diff review한다.

허용 경로

src/meeting_agent_workflow.py
web-demo/app.js

완료 조건

approve가 READY_FOR_EXPORT로 간다.
reject는 external_write=false와 REJECTED로 끝난다.

금지

Human Approval과 Interrupt: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Human Approval과 Interrupt 독립 리뷰

- | | | | |
|----|----------------------|------|---|
| 01 | Codex diff | ———— | Human Approval과 Interrupt 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | ———— | approve가 READY_FOR_EXPORT로 간다. |
| 03 | Claude review | ———— | review_decision.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | ———— | Human Approval과 Interrupt의 두 LLM 의견과 test는 자동 승인이 아니다 |

Human Approval과 Interrupt 정상 시연

- 01 — 열기: `src/langgraph_lab.py`
- 02 — 구현 확인: `src/meeting_agent_workflow.py`
- 03 — 실행: `python -m src.langgraph_lab --decision all`
- 04 — 성공 신호: `approve`가 `READY_FOR_EXPORT`로 간다.

Human Approval과 Interrupt 실행 명령

```
$ python -m src.langgraph_lab --decision all
```

실행 전 확인

Human Approval과 Interrupt: repository root · Python 3.12 · .env 미출력

06_review_decision.json 실행 결과

```
{
  "approve": {
    "status": "PUBLISHED",
    "external_write": true,
    "publisher_mode": "fake"
  },
  "reject": {
    "status": "BLOCKED",
    "target": {
      "repository": "classroom-owner/agent-sandbox",
      "number": 17,
      "head_sha": "7c83e8fce9a24d90b6d4a6c9123aa90817b65b1a"
    },
    "body": "## 교육용 코드 리뷰 초안\n\n- **P0** · 검증되지 않은 문자열이 코드로 실행될 수 있음*\n  - 위치: `src/payment_job.py:9`\n  - 근거: `result = eval(command)`\n  - 제안: 허용된 명령어나 파서만 호출하는 명시적 registry로 교체하세요.\n- **P1** · 외부 쓰기 전에 사람 승인 경계가 보이지 않음*\n  - 위치: `src/payment_job.py:11`\n  - 근거: `requests.post(\"https://example.invalid/jobs\", json=payload)`\n  - 제안: dry-run payload, 대상 확인, human_approved 조건을 쓰기 호출 앞에 두세요.\n- **P2** · 넓은 예외 처리가 실패 계약을 지움*\n  - 위치: `src/payment_job.py:12`\n  - 근거: `except Exception`\n  - 제안: 복구 가능한 예외만 잡고 구조화된 error_code를 반환하세요.\n\n자동 게시하지 않았습니다. 대상과 내용을 사람이 확인해야 합니다.",
    "idempotency_key": "c97ba39c959d7fda585b86c3",
    "human_approval_required": true,
    "external_write": false,
    "error_code": "HUMAN_APPROVAL_REQUIRED",
    "remote_result": null
  }
}
```

Human Approval과 Interrupt 실패·복구 시연

재현

**reject는 external_write=false와
REJECTED로 끝난다.**

복구

**interrupt를 workflow event로 두고
같은 thread_id와 JSON decision으로
resume한다.**

확인: reject는 external_write=false와 REJECTED로 끝난다.

Human Approval과 Interrupt 소프트웨어 제작

열 파일 `src/meeting_agent_workflow.py`
 `web-demo/app.js`

작업 범위 Human Approval과 Interrupt 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 approve가 READY_FOR_EXPORT로 간다.
 review_decision.json 저장

Human Approval과 Interrupt Notebook 셀

materials/day4/day4_service_lab.ipynb · 6차시

```
rejected = publish_review_comment(
    plan=approval_plan, human_approved=False, publisher=fake_publisher, store=InMemoryIdempotencyStore(),
)
human_decisions = {
    "approve": {"status": first["status"], "external_write": first["external_write"], "publisher_mode": "fake"},
    "reject": rejected,
}
assert rejected["error_code"] == "HUMAN_APPROVAL_REQUIRED"
save_json("06_review_decision.json", human_decisions)
human_decisions
```

Human Approval과 Interrupt Terminal 실행

```
$ python -m src.langgraph_lab --decision all
```

저장 위치

```
output/course-labs/day4/06_review_decision.json
```

approve가 READY_FOR_EXPORT로 간다.
실패 시 review_decision.json의 status·error_code를 먼저 확인

Human Approval과 Interrupt 실패 증거

reject는 external_write=false와 REJECTED로 끝난다.

```
result_file = "output/course-labs/day4/06_review_decision.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

review_decision.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Human Approval과 Interrupt 정상 Case

NORMAL

approve가 READY_FOR_EXPORT로 간다.

- 01 — src/langgraph_lab.py 입력과 command가 기록됐는가
- 02 — review_decision.json schema가 validate되는가
- 03 — Human Approval과 Interrupt focused test가 실제로 통과했는가

Human Approval과 Interrupt 경계 Case

BOUNDARY

reject는 external_write=false와 REJECTED로 끝난다.

- 01 — Human Approval과 Interrupt: raw traceback 대신 stable error_code가 남는가
- 02 — review_decision.json: 외부 쓰기·자동 메일이 false인가
- 03 — reject는 external_write=false와 REJECTED로 끝난다. 재현이 가능한가

Human Approval과 Interrupt 현업 적용

PR comment·메일·ticket 생성 전에 동일 승인 UI 사용

- 01 — Human Approval과 Interrupt은 합성·비식별 sample로 먼저 실행
- 02 — PR comment·메일·ticket 생성 전에 동일 승인 UI 사용의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — review_decision.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_decision.json을 운영 검토 자료로 사용

Human Approval과 Interrupt 포트폴리오 적용

Human Approval과 Interrupt · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — interrupt를 일반 Exception으로 잡아 자동 진행한다.의 사용자 영향을 한 문장으로 설명
- 02 — Human Approval과 Interrupt 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 승인 node 전에 외부 쓰기 호출이 존재하지 않는지 diff review한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — review_decision.json과 README 재현 절차를 제출

Human Approval과 Interrupt 운영 점검

품질

무엇을 approve가 READY_FOR_EXPORT로 간다.로 정의하는가

복구

interrupt를 일반 Exception으로 잡아 자동 진행한다.
발생 시 review_decision.json에서 원인 상태를 확인

안전

Human Approval과 Interrupt의 사람 승인과
external_write=false 위치

관측

review_decision.json에서 어떤 status·error_code를
보는가

review_decision.json 실행 증거

- 01 — 설명: Human Approval과 Interrupt이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langgraph_lab --decision all`
- 03 — 증거: review_decision.json과 정상·실패 test 결과가 남아 있다.

다음 차시: Dry-run Comment Payload

DAY 4 · 7차시

15:50-16:40 (쉬는 시간 17:30-18:00)

Dry-run Comment Payload

Dry-run · Payload · review_comment_plan.json

진행	사용 파일	확인할 결과
Dry-run Comment Payload 강의 12분 · 강사 시연 10분 · Codex 소프트웨어 제작 23분 · 실행 확인 5분	src/course_services/github_service.py src/course_services/contracts.py	review_comment_plan.json

7차시 입력과 결과

입력	————	review_decision.json
이번 차시	————	Dry-run Comment Payload
결과	————	review_comment_plan.json
다음 연결	————	Sandbox GitHub 게시

Dry-run Comment Payload 필요성

대충 만든 preview는 실제 API payload와 달라 승인 증거가 되지 못한다.

report validate에서 review_comment_plan.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: review_comment_plan.json

Dry-run Comment Payload 학습 결과

- 01 — PR comment body·target·idempotency key를 local JSON으로 만든다.
- 02 — dry_run=true인데 publisher가 호출된다. 왜 위험한지 설명한다.
- 03 — review_comment_plan.json에서 정상과 실패 상태를 직접 확인한다.

Dry-run Comment Payload 용어

용어	이 수업에서 쓰는 뜻
Dry-run	외부 쓰기 없이 동일 payload 생성
Payload	API에 보낼 구조화 데이터
Audit	누가 무엇을 승인했는지 기록
External write	외부 시스템 상태를 바꾸는 호출

review_comment_plan.json 입출력

INPUT

src/course_services/github_service.py

OUTPUT

review_comment_plan.json

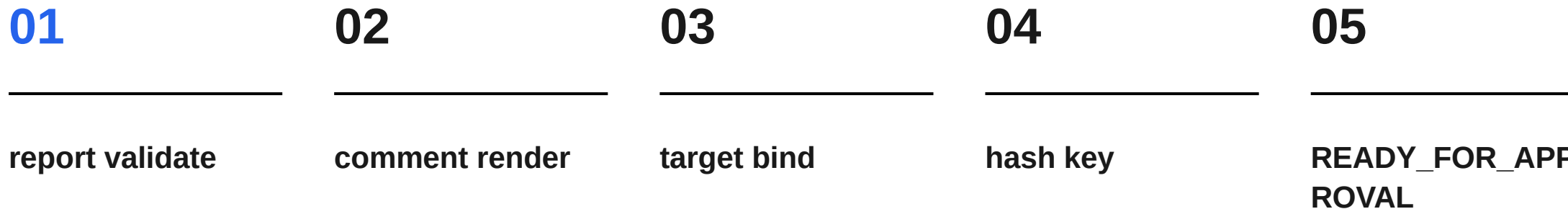
PROCESS

report validate → comment render → target bind

VERIFY

payload에 repo·PR·SHA·body·key가 있다.
dry-run plan은 approval=true여도 publish되지 않는다.

Dry-run Comment Payload 실행 흐름



마지막 판단은 review_comment_plan.json에서 사람이 확인합니다.

review_comment_plan.json 정상·실패 계약

정상	——	payload에 repo·PR·SHA·body·key가 있다.
경계	——	dry-run plan은 approval=true여도 publish되지 않는다.
외부 쓰기	——	Dry-run Comment Payload: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·review_comment_plan.json

Dry-run Comment Payload 파일 지도

입력·fixture

`src/course_services/github_service.py`

구현 코드

`src/course_services/contracts.py`

검증·test

`tests/test_course_services.py`

따라하기 notebook

`materials/day4/day4_service_lab.ipynb`

Dry-run Comment Payload 개념·코드

Dry-run

외부 쓰기 없이 동일 payload 생성

```
src/course_services/github_service.py
```

Audit

누가 무엇을 승인했는지 기록

```
tests/test_course_services.py
```

Payload

API에 보낼 구조화 데이터

```
src/course_services/contracts.py
```

External write

외부 시스템 상태를 바꾸는 호출

```
materials/day4/day4_service_lab.ipynb
```

Dry-run Comment Payload 선택 기준

구분	역할	이번 차시의 판단 기준
Dry-run	report validate 단계의 입력 기준	결정론 test로 먼저 확인
Payload	comment render 단계의 교체 경계	adapter 경계를 확인
Audit	target bind 단계의 운영 조건	비용·지연·권한을 확인
External write	hash key 단계의 판단 기록	사람 판단과 운영 기록을 확인

Dry-run Comment Payload 대표 실패

실패

dry_run=true인데 publisher가 호출된다.

사용자 영향

**dry_run=true인데 publisher가 호출된다.
상태에서는 review_comment_plan.json을
운영 판단에 사용할 수 없다.**

Dry-run Comment Payload 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

Dry-run Comment Payload 복구 경로

- 01 — 탐지: dry-run plan은 approval=true여도 publish되지 않는다.
- 02 — 중단: dry_run=true인데 publisher가 호출된다.
- 03 — 복구: DRY_RUN_CANNOT_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.
- 04 — 증거: review_comment_plan.json과 test 결과를 함께 남긴다.

Dry-run Comment Payload Codex 검증 루프

src/course_services/
contracts.py의 책임을 찾고
review_comment_plan.json
계약을 focused test로 고정합니다.

Dry-run Comment Payload
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

WORKFLOW LAB

PipelineReviewResult

03

COMMAND RESUME

같은 thread의 최종 상태를 확인합니다.

TERMINAL STATUS

REJECTED

외부로 내보내지 않고 검토 상태로 종료합니다.

RELEASE GATE

HOLD

Schema: 근거·사람 결정·외부 쓰기 금지를 검사합니다.

SIDE EFFECT

LOCAL JSON ONLY

메일·결제·고객 데이터 변경을 수행하지 않습니다.

AUDIT EVENTS

1. validate

REVIEW_REQUIRED

2. review

reject

3. rejected

OUTPUT

내 결정이 담긴 JSON

브라우저 Demo 결과를 파일로 저장해 Git commit 또는 포트폴리오 증거로 사용할 수 있습니다.

Dry-run Comment Payload Codex 작업 명세

목표

dry-run 경계를 건드리는 변경에는 boundary test 없이는 완료하지 않는다.

허용 경로

src/course_services/contracts.py
tests/test_course_services.py

완료 조건

payload에 repo·PR·SHA·body·key가 있다.
dry-run plan은 approval=true여도 publish되지 않는다.

금지

Dry-run Comment Payload: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Dry-run Comment Payload 독립 리뷰

- | | | | |
|----|----------------------|------|---|
| 01 | Codex diff | ———— | Dry-run Comment Payload 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | ———— | payload에 repo·PR·SHA·body·key가 있다. |
| 03 | Claude review | ———— | review_comment_plan.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | ———— | Dry-run Comment Payload의 두 LLM 의견과 test는 자동 승인이 아니다 |

Dry-run Comment Payload 정상 시연

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `src/course_services/contracts.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k dry_run`
- 04 — 성공 신호: payload에 `repo·PR·SHA·body·key`가 있다.

Dry-run Comment Payload 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k dry_run
```

실행 전 확인

Dry-run Comment Payload: repository root · Python 3.12 · .env 미출력

07_review_comment_plan.json 실행 결과

```
{
  "plan": {
    "status": "DRY_RUN",
    "target": {
      "repository": "classroom-owner/agent-sandbox",
      "number": 17,
      "head_sha": "7c83e8fce9a24d90b6d4a6c9123aa90817b65b1a"
    },
    "body": "## 교육용 코드 리뷰 초안\n\n- **P0 · 검증되지 않은 문자열이 코드로 실행될 수 있음**\n  - 위치: `src/payment_job.py:9`\n  - 근거: `result = eval(command)`\n  - 제안: 허용된 명령이나 파서만 호출하는 명시적 registry로 교체하세요.\n\n- **P1 · 외부 쓰기 전에 사람 승인 경계가 보이지 않음**\n  - 위치: `src/payment_job.py:11`\n  - 근거: `requests.post(\"https://example.invalid/jobs\", json=payload)`\n  - 제안: dry-run payload, 대상 확인, human_approved 조건을 쓰기 호출 앞에 두세요.\n\n- **P2 · 넓은 예외 처리가 실패 계약을 지움**\n  - 위치: `src/payment_job.py:12`\n  - 근거: `except Exception:`\n  - 제안: 복구 가능한 예외만 잡고 구조화된 error_code를 반환하세요.\n\n자동 게시하지 않았습니다. 대상과 내용을 사람이 확인해야 합니다.",
    "idempotency_key": "c97ba39c959d7fda585b86c3",
    "human_approval_required": true,
    "external_write": false,
    "error_code": null,
    "remote_result": null
  },
  "publish_attempt": {
    "status": "BLOCKED",
    "target": {
      "repository": "classroom-owner/agent-sandbox",
      "number": 17,
      "head_sha": "7c83e8fce9a24d90b6d4a6c9123aa90817b65b1a"
    },
    "body": "## 교육용 코드 리뷰 초안\n\n- **P0 · 검증되지 않은 문자열이 코드로 실행될 수 있음**\n  - 위치: `src/payment_job.py:9`\n  - 근거: `result = eval(command)`\n  - 제안: 허용된 명령이나 파서만 호출하는 명시적 registry로 교체하세요.\n\n- **P1 · 외부 쓰기 전에 사람 승인 경계가 보이지 않음**\n  - 위치: `src/payment_job.py:11`\n  - 근거: `requests.post(\"https://example.invalid/jobs\", json=payload)`\n  - 제안: dry-run payload, 대상 확인, human_approved 조건을 쓰기 호출 앞에 두세요.\n\n- **P2 · 넓은 예외 처리가 실패 계약을 지움**\n  - 위치: `src/payment_job.py:12`\n  - 근거: `except Exception:`\n  - 제안: 복구 가능한 예외만 잡고 구조화된 error_code를 반환하세요.\n\n자동 게시하지 않았습니다. 대상과 내용을 사람이 확인해야 합니다.",
    "idempotency_key": "c97ba39c959d7fda585b86c3",
    "human_approval_required": true,
    ...
  }
}
```

Dry-run Comment Payload 실패·복구 시연

재현

**dry-run plan은 approval=true여도
publish되지 않는다.**

복구

**DRY_RUN_CANNOT_PUBLISH를
contract로 고정하고 fake publisher
call 수를 assert한다.**

확인: dry-run plan은 approval=true여도 publish되지 않는다.

Dry-run Comment Payload 소프트웨어 제작

열 파일	<code>src/course_services/contracts.py</code> <code>tests/test_course_services.py</code>
작업 범위	Dry-run Comment Payload 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	payload에 repo·PR·SHA·body·key가 있다. review_comment_plan.json 저장

Dry-run Comment Payload Notebook 셀

materials/day4/day4_service_lab.ipynb · 7차시

```
dry_run_plan = prepare_review_comment(report=report, target=target, dry_run=True)
blocked_dry_run = publish_review_comment(
    plan=dry_run_plan, human_approved=True, publisher=fake_publisher, store=InMemoryIdempotencyStore(),
)
assert dry_run_plan["external_write"] is False
assert blocked_dry_run["error_code"] == "DRY_RUN_CANNOT_PUBLISH"
save_json("07_review_comment_plan.json", {"plan": dry_run_plan, "publish_attempt": blocked_dry_run})
{"status": dry_run_plan["status"], "publish_attempt": blocked_dry_run["error_code"]}
```

Dry-run Comment Payload Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k dry_run
```

저장 위치

output/course-labs/day4/07_review_comment_plan.json

payload에 repo·PR·SHA·body·key가 있다.
실패 시 review_comment_plan.json의 status·error_code를 먼저 확인

Dry-run Comment Payload 실패 증거

dry-run plan은 approval=true여도 publish되지 않는다.

```
result_file = "output/course-labs/day4/07_review_comment_plan.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

review_comment_plan.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Dry-run Comment Payload 정상 Case

NORMAL

payload에 repo·PR·SHA·body·key가 있다.

- 01 — src/course_services/github_service.py 입력과 command가 기록됐는가
- 02 — review_comment_plan.json schema가 validate되는가
- 03 — Dry-run Comment Payload focused test가 실제로 통과했는가

Dry-run Comment Payload 경계 Case

BOUNDARY

dry-run plan은 approval=true여도 publish되지 않는다.

- 01 — Dry-run Comment Payload: raw traceback 대신 stable error_code가 남는가
- 02 — review_comment_plan.json: 외부 쓰기·자동 메일이 false인가
- 03 — dry-run plan은 approval=true여도 publish되지 않는다. 재현이 가능한가

Dry-run Comment Payload 현업 적용

운영자가 Slack/PR 화면에서 실제 게시 내용을 사전 검토

- 01 — Dry-run Comment Payload은 합성·비식별 sample로 먼저 실행
- 02 — 운영자가 Slack/PR 화면에서 실제 게시 내용을 사전 검토의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — review_comment_plan.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_comment_plan.json을 운영 검토 자료로 사용

Dry-run Comment Payload 포트폴리오 적용

Dry-run Comment Payload · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — dry_run=true인데 publisher가 호출된다.의 사용자 영향을 한 문장으로 설명
- 02 — Dry-run Comment Payload 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — dry-run 경계를 건드리는 변경에는 boundary test 없이는 완료하지 않는다.의 diff와 직접 검토한 test 증거를 구분
- 04 — review_comment_plan.json과 README 재현 절차를 제출

Dry-run Comment Payload 운영 점검

품질

무엇을 payload에 repo·PR·SHA·body·key가 있다.로 정의하는가

복구

dry_run=true인데 publisher가 호출된다. 발생 시 review_comment_plan.json에서 원인 상태를 확인

안전

Dry-run Comment Payload의 사람 승인과 external_write=false 위치

관측

review_comment_plan.json에서 어떤 status·error_code를 보는가

review_comment_plan.json 실행 증거

- 01 — 설명: Dry-run Comment Payload이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k dry_run`
- 03 — 증거: review_comment_plan.json과 정상·실패 test 결과가 남아 있다.

다음 차시: **Sandbox GitHub** 게시

DAY 4 · 8차시

16:40-17:30 (쉬는 시간 17:30-18:00)

Sandbox GitHub 게시

Sandbox repo · Publisher · day4_audit_record.json

진행

Sandbox GitHub 게시
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

materials/day4/
day4_service_lab.ipynb
src/course_services/github_service.py

확인할 결과

day4_audit_record.json

8차시 입력과 결과

입력 ————— review_comment_plan.json

이번 차시 ————— **Sandbox GitHub 게시**

결과 ————— day4_audit_record.json

다음 연결 ————— 17:30-18:00 쉬는 시간

Sandbox GitHub 게시 필요성

교육 실습이 타인 저장소나 업무 repo를 오염시키면 안 된다.

sandbox 확인에서 day4_audit_record.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: day4_audit_record.json

Sandbox GitHub 게시 학습 결과

- 01 — 선택적으로 승인된 comment 한 건을 게시하고 audit ID를 남긴다.
- 02 — 교육 코드에 live publisher와 token scope를 기본 포함한다. 왜 위험한지 설명한다.
- 03 — day4_audit_record.json에서 정상과 실패 상태를 직접 확인한다.

Sandbox GitHub 게시 용어

용어	이 수업에서 쓰는 뜻
Sandbox repo	실습 전용 본인 저장소
Publisher	외부 API write adapter
Remote ID	게시 결과 식별자
Rollback	comment 삭제 또는 commit revert

day4_audit_record.json 입출력

INPUT

materials/day4/day4_service_lab.ipynb

OUTPUT

day4_audit_record.json

PROCESS

sandbox 확인 → dry-run 검토 → 사람 승인

VERIFY

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

Sandbox GitHub 게시 실행 흐름

01

sandbox 확인

02

dry-run 검토

03

사람 승인

04

1회 POST

05

response ID 기록

마지막 판단은 day4_audit_record.json에서 사람이 확인합니다.

day4_audit_record.json 정상·실패 계약

정상	————	fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
경계	————	publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.
외부 쓰기	————	Sandbox GitHub 게시: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	————	status·error_code·입력 식별자·day4_audit_record.json

Sandbox GitHub 게시 파일 지도

입력·fixture

materials/day4/day4_service_lab.ipynb

구현 코드

src/course_services/github_service.py

검증·test

.env.sample

따라하기 notebook

AGENTS.md

Sandbox GitHub 게시 개념·코드

Sandbox repo

실습 전용 본인 저장소

```
materials/day4/day4_service_lab.ipynb
```

Remote ID

게시 결과 식별자

```
.env.sample
```

Publisher

외부 API write adapter

```
src/course_services/github_service.py
```

Rollback

comment 삭제 또는 commit revert

```
AGENTS.md
```

Sandbox GitHub 게시 선택 기준

구분	역할	이번 차시의 판단 기준
Sandbox repo	sandbox 확인 단계의 입력 기준	결정론 test로 먼저 확인
Publisher	dry-run 검토 단계의 교체 경계	adapter 경계를 확인
Remote ID	사람 승인 단계의 운영 조건	비용·지연·권한을 확인
Rollback	1회 POST 단계의 판단 기록	사람 판단과 운영 기록을 확인

Sandbox GitHub 게시 대표 실패

실패

교육 코드에 live publisher와 token scope를 기본 포함한다.

사용자 영향

교육 코드에 live publisher와 token scope를 기본 포함한다. 상태에서는 day4_audit_record.json을 운영 판단에 사용할 수 없다.

Sandbox GitHub 게시 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

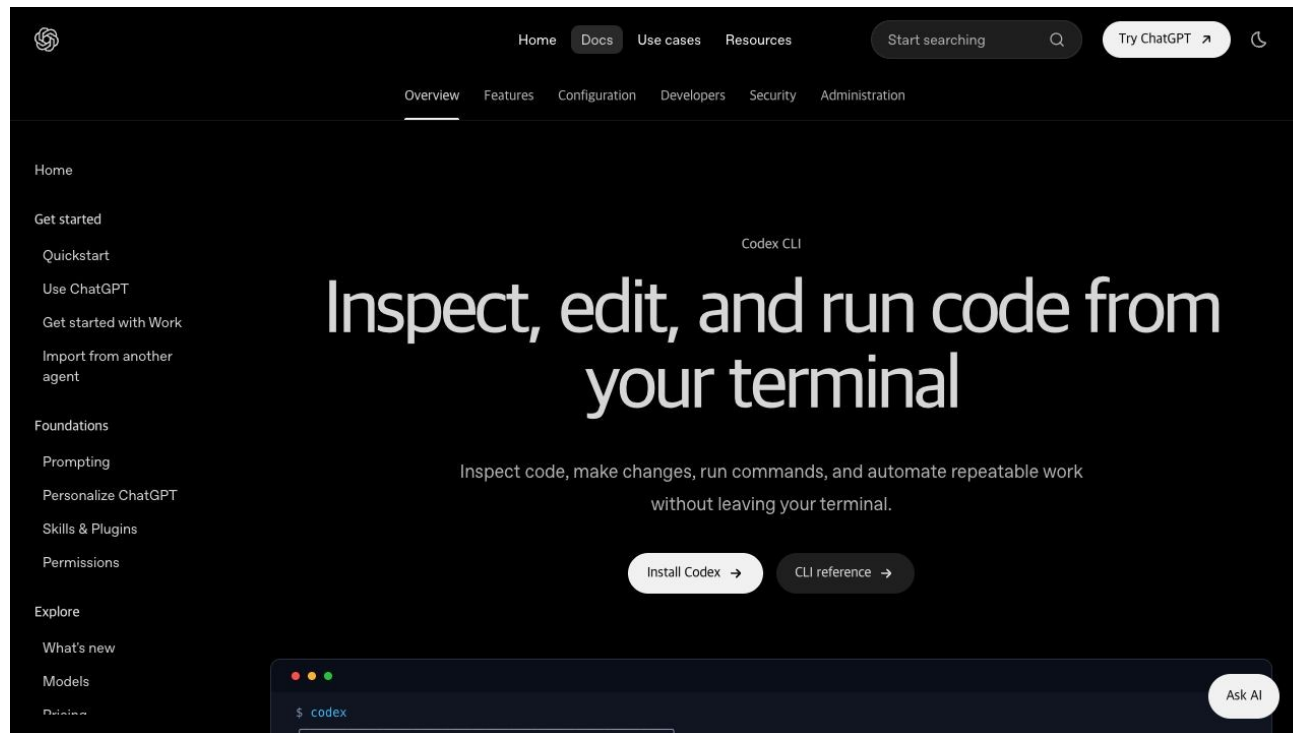
Sandbox GitHub 게시 복구 경로

- 01 — 탐지: publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.
- 02 — 중단: 교육 코드에 live publisher와 token scope를 기본 포함한다.
- 03 — 복구: repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.
- 04 — 증거: day4_audit_record.json과 test 결과를 함께 남긴다.

Sandbox GitHub 게시 Codex 검증 루프

`src/course_services/github_service.py`의 책임을 찾고
`day4_audit_record.json` 계약을
`focused test`로 고정합니다.

Sandbox GitHub 게시
저장소 이해 → 허용 범위 변경
→ `focused test` → `diff` 리뷰
→ 사람 merge



Sandbox GitHub 게시 Codex 작업 명세

목표

PR 생성·리뷰·test 수정은 돕되 merge와 외부 게시 결정은 사람에게 남긴다.

허용 경로

src/course_services/github_service.py
.env.sample

완료 조건

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

금지

Sandbox GitHub 게시: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Sandbox GitHub 게시 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | Sandbox GitHub 게시 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | fake publisher가 한 번 호출되고 remote ID가 audit에 남는다. |
| 03 | Claude review | —— | day4_audit_record.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | Sandbox GitHub 게시의 두 LLM 의견과 test는 자동 승인이 아니다 |

Sandbox GitHub 게시 정상 시연

- 01 — 열기: `materials/day4/day4_service_lab.ipynb`
- 02 — 구현 확인: `src/course_services/github_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py`
- 04 — 성공 신호: fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.

Sandbox GitHub 게시 실행 명령

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

Sandbox GitHub 게시: repository root · Python 3.12 · .env 미출력

08_day4_audit_record.json 실행 결과

```
{
  "day": 4,
  "service": "PR Review Automation",
  "status": "SUCCESS",
  "decision": "READY_FOR_SANDBOX_REVIEW",
  "input": {
    "fixture": "data/day4_github/pr_fixture.json",
    "synthetic": true
  },
  "stages": [
    {
      "name": "target_validation",
      "status": "SUCCESS",
      "artifact": "pr_target.json"
    },
    {
      "name": "review_dry_run",
      "status": "DRY_RUN",
      "artifact": "review_comment_plan.json"
    },
    {
      "name": "human_approval",
      "status": "RECORDED",
      "artifact": "review_decision.json"
    },
    ...
  ]
}
```

Sandbox GitHub 게시 실패·복구 시연

재현

publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

복구

repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.

확인: publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

Sandbox GitHub 게시 소프트웨어 제작

열 파일	<code>src/course_services/github_service.py</code> <code>.env.sample</code>
작업 범위	Sandbox GitHub 게시 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	fake publisher가 한 번 호출되고 remote ID가 audit에 남는다. day4_audit_record.json 저장

Sandbox GitHub 게시 Notebook 셀

materials/day4/day4_service_lab.ipynb · 8차시

```
from src.course_services.course_demo import build_course_demo

day4_audit = build_course_demo(4, workspace_root=ROOT)
focused_test = run_command(sys.executable, "-m", "pytest", "-q", "tests/test_course_services.py", "-k", "github or
dry_run or idempotent")
day4_audit["focused_test"] = focused_test
assert day4_audit["external_write"] is False
assert focused_test["returncode"] == 0
save_json("08_day4_audit_record.json", day4_audit)
{"decision": day4_audit["decision"], "metrics": day4_audit["metrics"]}
```


Sandbox GitHub 게시 Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py
```

저장 위치

```
output/course-labs/day4/08_day4_audit_record.json
```

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
실패 시 day4_audit_record.json의 status·error_code를 먼저 확인

Sandbox GitHub 게시 실패 증거

publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

```
result_file = "output/course-labs/day4/08_day4_audit_record.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

day4_audit_record.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Sandbox GitHub 게시 정상 Case

NORMAL

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.

- 01 — materials/day4/day4_service_lab.ipynb 입력과 command가 기록됐는가
- 02 — day4_audit_record.json schema가 validate되는가
- 03 — Sandbox GitHub 게시 focused test가 실제로 통과했는가

Sandbox GitHub 게시 경계 Case

BOUNDARY

publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

- 01 — Sandbox GitHub 게시: raw traceback 대신 stable error_code가 남는가
- 02 — day4_audit_record.json: 외부 쓰기·자동 메일이 false인가
- 03 — publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다. 재현이 가능한가

Sandbox GitHub 게시 현업 적용

현업은 GitHub App과 protected branch, 구직자는 sandbox demo recording

- 01 — Sandbox GitHub 게시은 합성·비식별 sample로 먼저 실행
- 02 — 현업은 GitHub App과 protected branch, 구직자는 sandbox demo recording의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — day4_audit_record.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — day4_audit_record.json을 운영 검토 자료로 사용

Sandbox GitHub 게시 포트폴리오 적용

Sandbox GitHub 게시 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 교육 코드에 live publisher와 token scope를 기본 포함한다.의 사용자 영향을 한 문장으로 설명
- 02 — Sandbox GitHub 게시 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — PR 생성·리뷰·test 수정은 돕되 merge와 외부 게시 결정은 사람에게 남긴다.의 diff와 직접 검토한 test 증거를 구분
- 04 — day4_audit_record.json과 README 재현 절차를 제출

Sandbox GitHub 게시 운영 점검

품질

무엇을 fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.로 정의하는가

복구

교육 코드에 live publisher와 token scope를 기본 포함한다. 발생 시 day4_audit_record.json에서 원인 상태를 확인

안전

Sandbox GitHub 게시의 사람 승인과 external_write=false 위치

관측

day4_audit_record.json에서 어떤 status·error_code를 보는가

day4_audit_record.json 실행 증거

- 01 — 설명: Sandbox GitHub 게시이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: day4_audit_record.json과 정상·실패 test 결과가 남아 있다.

17:30-18:00 쉬는 시간